

Tensor Network Pretraining in Quantum Machine Learning

Hanskhel Muttur

August 17, 2026



MSc in Theoretical Physics
The University of Edinburgh
2026

Abstract

Variational quantum circuits (VQCs) can be used to perform machine learning tasks on quantum computers. However, training these models can be expensive, particularly because evaluating quantum circuits and their gradients may require many circuit executions.

In this work, we develop a new approach for classically pretraining VQCs with matrix product states (MPS), based on direct parameter transfer from the MPS to the VQC, eliminating the need for an intermediate decomposition algorithm. This is achieved by parameterising both the MPS and the VQC using the same parent matrices.

We first demonstrate the method using dense $SO(n)$ matrices and test it on a Bars-and-Stripes binary classification task. The results show that, when the MPS has sufficient expressivity, pretraining can place the VQC close to a good solution before VQC optimisation begins.

We then introduce a more efficient parameterisation based on R_y and controlled- R_y gates. In this construction, the parent matrices are generated directly from a structured elementary-gate circuit, avoiding the need to decompose general dense multi-qubit transformations before implementation on quantum hardware, while preserving the direct MPS-to-VQC parameter transfer. Our results show that the main pretraining behaviour can still be obtained with this more structured parameterisation.

Declaration

I declare that this dissertation was written entirely by myself.

The original idea of investigating tensor-network pretraining for variational quantum circuits, and in particular the direction of using direct parameterisation between MPS and VQC, was proposed by my supervisor, Bryan Coyle. The specific parameterisations, numerical implementations, experiments, and analysis developed in this dissertation were carried out by me under his supervision.

Chapter 2 provides the background necessary to understand the project. It discusses quantum computing, machine learning, tensor networks, and quantum machine learning, and is based on previous work which is cited throughout the chapter.

Chapters 3 and 4 contain the main original work of this project. In Chapter 3, I introduce and test a parameterisation which allows the parameters of an MPS to be transferred directly to a VQC without requiring an intermediate decomposition algorithm. In Chapter 4, I introduce a more efficient parameterisation based on R_y and controlled- R_y gates and investigate whether the same pretraining behaviour can still be obtained.

The numerical experiments, analysis, and figures presented in these chapters were carried out by me.

The numerical implementation was written in Python. I used PyTorch for tensor operations and optimisation, PennyLane for the simulation and differentiation of quantum circuits, Torchvision for dataset utilities, and Matplotlib for producing figures. The MPS and VQC models, the direct parameter-transfer procedure, and the training routines used in the project were implemented by me.

Artificial intelligence tools were used during the project as an aid for brainstorming, checking and correcting mistakes, and assisting with parts of the code debugging.

Chapter 5 contains the conclusion of the project, while Appendices A and B provide additional details and derivations of the MPS and MPS-to-VQC constructions used in the main text.

All previous work and external sources used in this dissertation are cited where appropriate.

The code used for the numerical implementation and experiments is available at <https://github.com/Hans0408/QML>.

Personal Statement

I spent the first two weeks of the project implementing a simple MPS model for machine learning and becoming familiar with how MPS models can be used for classification. During this time, I also became more familiar with PyTorch and the `einsum` functionality used for tensor contractions.

During this time, I also started studying how the mapping between an MPS and a variational quantum circuit works. This helped me understand the main idea behind the project and what would be required to transfer parameters from a classically trained MPS to a VQC. During weeks 3 and 4, I developed the baseline code for the project. I spent a large amount of time testing and debugging the implementation to make sure that the MPS and VQC behaved as expected. In particular, I checked the construction of the MPS tensors, the corresponding quantum gates, and the outputs produced by the two models. This stage was important for making sure that the parameterisation and parameter-transfer procedure were implemented correctly before carrying out the main experiments.

In weeks 5 and 6, I finalised the complete MPS-to-VQC pipeline and designed the numerical experiments used in this dissertation. I implemented the different parameterisations considered in the project, set up the Bars-and-Stripes classification task, and ran the experiments comparing pretrained and non-pretrained VQCs. I also investigated the effect of changing the bond dimension, using the more efficient circuit-based parameterisation, and adding additional trainable layers to the VQC after parameter transfer.

For the final four weeks of the project, from weeks 7 to 10, I focused mainly on writing the dissertation. During this period I organised and analysed the numerical results, produced the figures used in the report, and developed the explanation of the MPS-to-VQC parameter-transfer method. I also carried out the additional-gate experiments inspired by [36], where extra trainable gates are added to the VQC after parameter transfer. I continued checking parts of the implementation and results where necessary while writing the final report.

Acknowledgments

I would like to thank my supervisor, Mr Brian Coyle, who, beyond being an incredible supervisor, is also a wonderful person. I am deeply grateful for his patience, his advice, and everything he has taught me.

To my Gao, who believes in me more than anyone.

To Sneha, for her unfailing support.

To my parents, Liseby and Moïse, who have sacrificed so much for me.

Contents

1	Introduction	2
2	Background	4
2.1	Quantum Computing	4
2.1.1	Qubits and Quantum States	4
2.1.2	Quantum Circuit	7
2.1.3	Data Encoding	9
2.2	Machine Learning	12
2.2.1	The task T	12
2.2.2	Data Preparation and Dataset Partitioning	13
2.2.3	Feature mapping	14
2.2.4	Training algorithm	15
2.2.5	Generalisation and Model Complexity	20
2.3	Tensor Network	22
2.3.1	Matrix Product States (MPS)	23
2.3.2	Tensor Networks for Machine Learning	24
2.4	Quantum Machine Learning	27
2.4.1	Variational Quantum Models	27
2.4.2	Challenges and Limitations	28
3	Dense $SO(n)$ parameterisation	31
3.1	MPS to VQC	31
3.2	Experiments	32
3.2.1	Effect of the bond dimension	34
3.2.2	Adding trainable layers to the pretrained VQC	37
4	Efficient parameterisation	40

4.1	Experiments	42
4.1.1	Additional trainable VQC layers	44
5	Conclusion	47
A	Three-Site MPS Example	49
B	MPS to VQC	56
B.1	Conversion of the Three-Site MPS into a Quantum Circuit	56
B.1.1	Isometric form of the MPS tensors	56
B.1.2	Conversion of the first tensor	57
B.1.3	Rewriting the middle contraction	58
B.1.4	Orthogonal completion of the middle tensor	60
B.1.5	Action of the middle quantum gate	61
B.1.6	Quantum-state interpretation of the middle contraction	62
B.1.7	Conversion of the final tensor	62
B.1.8	The complete VQC contains additional amplitudes	64
B.1.9	Relation between the MPS and VQC outputs	65

List of Tables

2.1	Some common quantum gates. The table presents the gate notation, matrix representation, and quantum-circuit representation. Here, i denotes the imaginary unit.	8
-----	---	---

List of Figures

2.1	Quantum circuit for preparing the three-qubit GHZ state.	8
2.2	Example of basis encoding of the binary vector $\mathbf{x} = (1, 0, 1)$	9
2.3	Conceptual illustration of binary classification in the feature space. Each point represents an image of a handwritten digit (0 or 1) from the MNIST dataset. The classifier learns a decision boundary that separates the two classes. For visualisation purposes, only two features are shown, whereas the original MNIST images contain 784 pixel features.	13
2.4	Illustration of the MNIST data used for binary classification. Each 28×28 grayscale image of a handwritten digit is represented as a 784-dimensional feature vector and assigned a label $y \in \{0, 1\}$	14
2.5	Illustration of a nonlinear feature map. In the original two-dimensional input space, the inner and outer classes are not linearly separable. After mapping the data into a higher-dimensional feature space, the two classes can be separated by a linear decision plane.	15
2.6	Illustration of gradient descent in parameter space. The left panel shows the optimisation trajectory on the loss surface $\mathcal{L}(\theta_1, \theta_2)$, while the right panel shows the same trajectory projected onto the corresponding contour plot.	19
2.7	Illustration of model complexity: (a) underfitting, where the model is too simple to capture the underlying pattern; (b) an appropriate fit, where the model captures the general trend; and (c) overfitting, where the model follows noise and small fluctuations in the training data.	22
2.8	Tensor-network representation of a six-site matrix product state. The original tensor T is factorised into a one-dimensional chain of lower-order tensors $A^{[1]}, \dots, A^{[6]}$, connected by internal bond indices.	23
2.9	Illustration of the bond dimensions of a six-site MPS. The bond index connecting tensors k and $k + 1$ has dimension χ_k , and the overall bond dimension is defined as $\chi = \max_k \chi_k$	23
2.10	Tensor-network representation of an MPS with an uncontracted output index t at the final tensor. The horizontal lines represent the bond indices connecting neighbouring tensors.	24

2.11	Product feature map used to encode an image for the tensor-network classifier. Each pixel p_i is mapped independently to the two-dimensional feature vector $\Phi(p_i)$, and the complete encoded input is the tensor product $\Phi(\mathbf{x}) = \Phi(p_1) \otimes \cdots \otimes \Phi(p_N)$	25
2.12	Illustration of tensor-network classification. The local feature vectors are contracted with the input physical indices of the MPO, while the output physical index remains uncontracted. For binary classification, this output index produces the two class logits z_0 and z_1	26
3.1	The four 2×2 Bars-and-Stripes patterns used to construct the classification dataset. Vertical-bar patterns are assigned to class 0, while horizontal-stripe patterns are assigned to class 1.	32
3.2	Illustration of the direct MPS-to-VQC mapping. The MPS tensors and the corresponding VQC gates are generated from the same parameterised orthogonal matrices. The MPS uses selected parts of these matrices, while the VQC uses the complete matrices as quantum gates. After classical MPS training, the optimised parameters are transferred directly to the corresponding VQC gates.	33
3.3	Training loss and test accuracy for the dense $SO(n)$ parameterisation with bond dimensions $\chi = 2, 4, 8$. The classically trained MPS is compared with the corresponding pretrained VQC and a non-pretrained VQC with the same circuit architecture. Epoch zero denotes the VQC performance immediately after initialisation, before any VQC optimisation is performed. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.	36
3.4	Training loss of the pretrained VQC with bond dimension $\chi = 4$ for different numbers of additional trainable layers $L = 0, 1, 2, 3$. The case $L = 0$ corresponds to the direct MPS-to-VQC mapping, while $L > 0$ introduces additional trainable two-qubit $SO(4)$ gates initialised close to the identity. All models therefore begin from approximately the same loss, while the additional layers allow the VQC to reduce the loss more rapidly during the first few optimisation epochs. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.	38
4.1	Example of two layers of the structured circuit used to generate an efficient parent matrix. Each layer consists of trainable single-qubit R_y rotations followed by nearest-neighbour controlled- R_y rotations. The direction of the controlled-rotation ladder alternates between successive layers.	41
4.2	Direct parameter-transfer construction for the efficient parameterisation. The structured circuit generates the parent matrix $U_{\text{eff}}(\boldsymbol{\theta})$. Selected rows or columns are used to construct the corresponding MPS tensor, while the complete matrix is used as the VQC gate. Both therefore depend on the same parameters $\boldsymbol{\theta}$	41

4.3	Training loss and test accuracy for the efficient parameterisation with $L_c = 2, 4, 6$ and fixed bond dimension $\chi = 4$. Results are shown for the classically trained MPS, the pretrained VQC, and the non-pretrained VQC with the same circuit architecture. Epoch zero denotes the VQC performance immediately after initialisation and before any VQC optimisation. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.	43
4.4	Training loss for the efficient parameterisation with $L_c = 6$ and additional trainable VQC layers $L = 0, 1, 2, 3$. All pretrained VQCs begin from approximately the same loss because the additional layers are initialised close to the identity. The additional layers reduce the loss more rapidly during the first few optimisation epochs, while all models eventually converge to a similar low-loss region. The inset highlights the early stages of training. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.	45

Chapter 1

Introduction

Quantum machine learning combines ideas from quantum computing and machine learning with the aim of using quantum computers to perform learning tasks. One approach is to use a parameterised quantum circuit as a machine-learning model. The parameters of the circuit are trained by minimising a loss function, in a similar way to the parameters of a classical model.

However, training variational quantum circuits can be difficult. One problem is the presence of barren plateaus, where the gradients of the loss function can become very small. Another problem is the computational cost of training on quantum hardware. Quantum measurements are probabilistic, so a circuit generally has to be executed many times to estimate its output. Calculating gradients can require even more circuit executions. As the number of parameters and optimisation steps increases, this can make VQC training expensive.

Work has previously been done to attenuate these issues, such as in [36], where Matrix Product States were used for classical pre-training. However, these classical pretraining relies on compilation procedures, such as SVD, to map MPS to VQC approximately. This leads us to the question addressed in this project: can we parameterise our MPS and VQC in such a way that we can perform direct parameter transfer between the two without the need for any compilation?

The novel part of this method is how the MPS is parameterised to allow direct parameter transfer to the VQC. This is done by using the same parent matrices to build the MPS tensors and VQC gates. We first study this idea using dense special orthogonal matrices in $SO(n)$ as parent matrices. We then test whether the classical MPS pretraining provides a useful VQC initialisation on a binary Bars-and-Stripes classification task. We also study how the quality of the pretraining depends on the bond dimension and whether additional trainable gates can be added after the parameter transfer as in [36].

The dense parameterisation provides a useful first demonstration of the method, but it also has an important limitation. As the matrices become larger, implementing the corresponding dense transformations can require a rapidly increasing number of elementary gates and therefore deep quantum circuits.

For this reason, we then introduce a more efficient parameterisation. Rather than con-

structing a general dense matrix, we generate each parent matrix from a structured circuit made of R_y and nearest-neighbour controlled- R_y gates. The MPS is therefore parameterised directly by an efficient quantum-circuit structure. This preserves the direct parameter transfer while giving the local transformations an explicit elementary-gate structure and avoiding the need for a separate dense-matrix decomposition.

The main aim of the experiments is therefore not only to compare final classification accuracy. More importantly, we want to determine whether the classically trained MPS can provide a better starting point for the VQC, so that less VQC optimisation is required to reach a good solution. We also investigate whether this behaviour is preserved when the dense parameterisation is replaced by the more efficient circuit structure.

Chapter 2

Background

In this chapter, we will provide a **non-exhaustive** recap of the important background knowledge and concepts that will be useful to understand the project. The goal is also to show how these different topics are inter-related and how they work together.

2.1 Quantum Computing

The format and content of this section have been greatly inspired by chapter 2 of the book: *A Hands-on Tutorial for Machine Learning Practitioners and Researchers* [1]

2.1.1 Qubits and Quantum States

Qubits

In classical computing, information is encoded in bits, which can be in two states: 0 and 1. Multiple bits can be used together to represent more information. For example, 3 bits represent $2^3 = 8$ possible states, from 000 to 111. For quantum computation, qubits have an analogous role to bits. They are used to represent information in a two-state quantum system, such as the spin states of a particle. Since qubits represent quantum states, they can be described mathematically using the formalism of quantum mechanics.

Single Qubit State

We can clearly see this when, mathematically, we write a qubit state as

$$|a\rangle = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \in \mathbb{C}^2. \quad (2.1)$$

or equivalently,

$$|a\rangle = a_1|0\rangle + a_2|1\rangle, \quad (2.2)$$

with

$$|a_1|^2 + |a_2|^2 = 1, \quad (2.3)$$

satisfying the normalisation condition, where a_1 and a_2 are the probability amplitudes. Therefore, if the qubit is measured in the computational basis, the probabilities of measuring 0 and 1 are $|a_1|^2$ and $|a_2|^2$, respectively.

In Equation (2.2), we use bra-ket notation [2], where

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad (2.4)$$

are the computational basis states.

Two-Qubit State

To understand a two-qubit state, it is first necessary to introduce the tensor product. The tensor product of two vectors is defined as

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \otimes \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 y_1 \\ x_1 y_2 \\ x_2 y_1 \\ x_2 y_2 \end{pmatrix}. \quad (2.5)$$

For two single-qubit states $|a\rangle$ and $|b\rangle$, their joint product state is

$$\begin{aligned} |a\rangle \otimes |b\rangle &= a_1 b_1 |0\rangle \otimes |0\rangle + a_1 b_2 |0\rangle \otimes |1\rangle \\ &\quad + a_2 b_1 |1\rangle \otimes |0\rangle + a_2 b_2 |1\rangle \otimes |1\rangle \in \mathbb{C}^4. \end{aligned} \quad (2.6)$$

The corresponding computational basis is

$$|0\rangle \otimes |0\rangle \equiv \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |0\rangle \otimes |1\rangle \equiv \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad (2.7)$$

$$|1\rangle \otimes |0\rangle \equiv \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad |1\rangle \otimes |1\rangle \equiv \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \quad (2.8)$$

We can also write

$$|a\rangle \otimes |b\rangle = |ab\rangle = |a\rangle |b\rangle. \quad (2.9)$$

Therefore, the tensor product provides a mathematical framework for describing the joint state of two quantum systems, including a system composed of two qubits.

However, not every two-qubit state can be written as a tensor product of two single-qubit states. In general, a two-qubit state can be written as

$$|\psi\rangle = c_{00} |00\rangle + c_{01} |01\rangle + c_{10} |10\rangle + c_{11} |11\rangle, \quad (2.10)$$

with

$$|c_{00}|^2 + |c_{01}|^2 + |c_{10}|^2 + |c_{11}|^2 = 1. \quad (2.11)$$

States which cannot be written as a tensor product of two single-qubit states are called entangled states.

Multi-Qubit State

We can generalise this to an N -qubit state, with $N > 2$. If $|\psi\rangle$ is an N -qubit state, we have

$$|\psi\rangle = \sum_{i=0}^{2^N-1} c_i |i\rangle \in \mathbb{C}^{2^N}, \quad (2.12)$$

with

$$\sum_{i=0}^{2^N-1} |c_i|^2 = 1. \quad (2.13)$$

Here, $|i\rangle$ represents one of the computational basis states of the N -qubit system.

Note: The dimension increases exponentially with the number of qubits. This will be important later when discussing the expressivity of quantum machine learning models.

Entanglement

The tensor product provides a vector space for representing multi-qubit states. In this 2^N -dimensional vector space, each multi-qubit state can be described by a vector. Entangled states are vectors that cannot be generated from the individual state vectors of the subsystems.

Equivalently, a state is entangled if it cannot be expressed as a tensor product of the states of its subsystems. If

$$|\psi\rangle \in \mathcal{H}_A \otimes \mathcal{H}_B, \quad (2.14)$$

then $|\psi\rangle$ is separable if there exist states $|\psi_A\rangle \in \mathcal{H}_A$ and $|\psi_B\rangle \in \mathcal{H}_B$ such that

$$|\psi\rangle = |\psi_A\rangle \otimes |\psi_B\rangle. \quad (2.15)$$

Otherwise, the state is entangled.

A typical example of an entangled multi-qubit state is the GHZ state [3],

$$|\text{GHZ}\rangle = \frac{1}{\sqrt{2}} \left(|0\rangle^{\otimes N} + |1\rangle^{\otimes N} \right). \quad (2.16)$$

For $N = 3$, the GHZ state is given by

$$|\text{GHZ}_3\rangle = \frac{1}{\sqrt{2}} (|000\rangle + |111\rangle). \quad (2.17)$$

Note. Quantum entanglement is important because it is one of the resources that can contribute to achieving computational advantages over classical computation [4].

2.1.2 Quantum Circuit

We would now like to interact with our qubits in order to perform computations. In classical computers, logical operations are performed on bits using logic gates, such as the NOT, AND, and OR gates.

Quantum Gates

In quantum computing, the role of classical logic gates is occupied by quantum gates. A quantum gate acting on k qubits is represented by a unitary operator in $U(2^k)$. When acting as part of an N -qubit system, the corresponding operator is embedded in the full 2^N -dimensional Hilbert space. Up to a physically irrelevant global phase, they may also be represented by operators in $SU(2^N)$. This is consistent with our knowledge of quantum mechanics, in which quantum states are transformed from one state to another by unitary operators.

The transformation of a quantum state $|a\rangle$ into a state $|\hat{a}\rangle$ by an arbitrary unitary operator U is given by

$$|\hat{a}\rangle = U |a\rangle. \quad (2.18)$$

This can be visualised using a quantum circuit diagram:

$$|a\rangle \text{ --- } \boxed{U} \text{ --- } |\hat{a}\rangle \quad (2.19)$$

Table 2.1 shows some elementary gates that are frequently used.

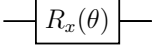
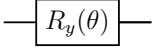
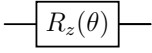
Gate	Matrix representation	Circuit representation
$R_X(\theta)$	$R_x(\theta) = \begin{pmatrix} \cos(\frac{\theta}{2}) & -i \sin(\frac{\theta}{2}) \\ -i \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$	
$R_Y(\theta)$	$R_y(\theta) = \begin{pmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$	
$R_Z(\theta)$	$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$	
Controlled-NOT (CNOT)	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	

Table 2.1: Some common quantum gates. The table presents the gate notation, matrix representation, and quantum-circuit representation. Here, i denotes the imaginary unit.

A quantum circuit is constructed by applying a sequence of quantum gates to one or more qubits. When gates act on different qubits at the same time, their joint action can be represented using the tensor product. For example, applying a single-qubit gate U to the first qubit of a two-qubit system while leaving the second qubit unchanged is represented by

$$U \otimes I_2. \quad (2.20)$$

When several gates are applied sequentially, the corresponding operators are multiplied in the same way as ordinary quantum-mechanical operators. The rightmost operator acts first. Therefore, if

$$U = U_3 U_2 U_1, \quad (2.21)$$

then U_1 acts first, followed by U_2 , and finally U_3 . Quantum-circuit diagrams represent the same sequence graphically, with the circuit read from left to right.

Example : If $U = (I_2 \otimes \text{CNOT})(\text{CNOT} \otimes I_2)(H \otimes I_4)$, then we will have the following circuit.

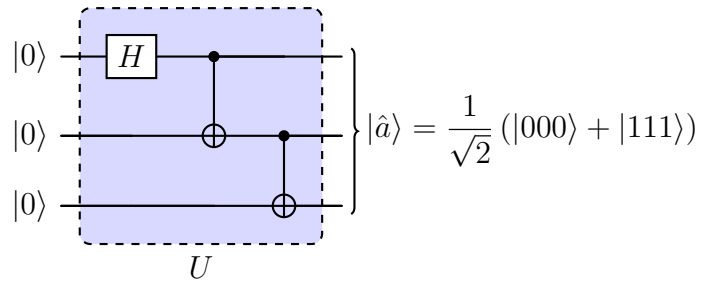


Figure 2.1: Quantum circuit for preparing the three-qubit GHZ state.

Universal Quantum Gate Set

In the previous example, the gate U was already decomposed into elementary gates. However, one may ask what happens for any arbitrary unitary operator $U \in SU(2^N)$. It has been proved that such an operator can be constructed from a restricted collection of elementary single-qubit and two-qubit gates. In other words, it is sufficient to use a universal set of gates to approximate to arbitrary precision a unitary operation. A common example of a universal gate set is $R_x(\theta), R_y(\theta), R_z(\theta)$, together with the $CNOT$ gate.

2.1.3 Data Encoding

Data encoding is part of the broader framework called *read-in protocols*, which refers to the process of encoding classical information into a quantum computer. This is particularly important as it allows quantum computers to solve problems involving classical data. We will consider three types of data encoding, namely amplitude encoding, angle encoding, and basis encoding.

Basis encoding

Basis encoding is a way to encode classical data that are in binary form. If we have a vector

$$\mathbf{x} = (x_0, \dots, x_i, \dots, x_{N-1}) \in \{0, 1\}^N, \quad (2.22)$$

then we can directly encode it into the quantum computational basis as

$$|\psi\rangle = |x_0, \dots, x_i, \dots, x_{N-1}\rangle. \quad (2.23)$$

As by default, the initial qubits are set to $|0\rangle$, we prepare this state by applying an X gate where the corresponding bit value is 1. We can write

$$|\psi\rangle = \bigotimes_{i=0}^{N-1} X^{x_i} |0\rangle. \quad (2.24)$$

Example: Consider the binary vector $\mathbf{x} = (1, 0, 1)$. Using basis encoding, this vector is encoded as $|\psi\rangle = |101\rangle$.

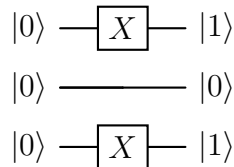


Figure 2.2: Example of basis encoding of the binary vector $\mathbf{x} = (1, 0, 1)$.

Amplitude encoding

Amplitude encoding is when data are encoded in the amplitudes of the quantum state.

If we have a vector

$$\mathbf{x} = (x_0, \dots, x_i, \dots, x_{2^N-1}) \in \mathbb{C}^{2^N}, \quad (2.25)$$

and if we normalise it as

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}, \quad (2.26)$$

where the ℓ_2 norm is defined as

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=0}^{2^N-1} |x_i|^2}, \quad (2.27)$$

we can prepare a quantum state as follows:

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \hat{x}_i |i\rangle \quad (2.28)$$

with $|i\rangle$ representing the N -qubit computational basis states.

Example : If we have a vector $\mathbf{x} = (x_0, x_1) \in \mathbb{R}^2$, with $x_0^2 + x_1^2 = 1$, we can use amplitude encoding to produce the quantum state $|\psi\rangle = x_0 |0\rangle + x_1 |1\rangle$.

This can be achieved by applying a rotation gate $U = R_y(\theta)$ to the initial state $|0\rangle$, where $\theta = 2 \arccos(x_0)$.

Angle encoding

Angle encoding uses rotation angles to encode data into quantum states.

If we have a vector $\mathbf{x} = (x_0, \dots, x_i, \dots, x_{N-1}) \in \mathbb{R}^N$, the encoded quantum state is

$$|\psi\rangle = \bigotimes_{i=0}^{N-1} R_\sigma(x_i) |0\rangle = \bigotimes_{i=0}^{N-1} \exp\left(-i \frac{x_i}{2} \sigma\right) |0\rangle, \quad (2.29)$$

where $\sigma \in \{X, Y, Z\}$.

Example: If we use R_y for the angle encoding of the vector $\mathbf{x} = (x_1, x_2)$, we have

$$R_y(x_i) |0\rangle = \cos\left(\frac{x_i}{2}\right) |0\rangle + \sin\left(\frac{x_i}{2}\right) |1\rangle. \quad (2.30)$$

Therefore, the encoded two-qubit state is

$$|\psi\rangle = R_y(x_1) |0\rangle \otimes R_y(x_2) |0\rangle. \quad (2.31)$$

Angle encoding offers a number of benefits compared to basis encoding and amplitude encoding. Basis encoding may require the same number of qubits as the input data

dimensionality, whereas amplitude encoding requires fewer qubits but has greater gate complexity to encode the data. Angle encoding provides a practical compromise by encoding each feature directly into the parameter of a rotation gate. For this reason, angle encoding circuits are generally simpler and shallower.

In quantum machine learning, angle encoding can be interpreted as a nonlinear feature map from the classical input space to a quantum Hilbert space. This makes it particularly convenient for constructing parameterised quantum feature maps, as discussed in Section 2.2.3.

Measurement

After a quantum circuit has prepared and manipulated a quantum state, a measurement is required to extract classical information from the system. In the computational basis, measuring an N -qubit state

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i |i\rangle \quad (2.32)$$

produces one of the computational basis states $|i\rangle$ with probability

$$p_i = |\alpha_i|^2, \quad (2.33)$$

where the probabilities satisfy

$$\sum_{i=0}^{2^N-1} |\alpha_i|^2 = 1. \quad (2.34)$$

According to the Born rule, the outcome of a quantum measurement is probabilistic. Consequently, a quantum circuit is generally executed multiple times, commonly referred to as *shots*, in order to estimate the probability distribution associated with the possible measurement outcomes.

More generally, information can be extracted from a quantum state through the expectation value of an observable O , given by

$$\langle O \rangle = \langle \psi | O | \psi \rangle. \quad (2.35)$$

Expectation values are particularly important in quantum machine learning, where they are often used as the classical output of a parameterised quantum circuit.

Example: consider the Pauli- Z observable,

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.36)$$

For a single-qubit state

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad (2.37)$$

with $|\alpha|^2 + |\beta|^2 = 1$, the expectation value is

$$\langle Z \rangle = \langle \psi | Z | \psi \rangle. \quad (2.38)$$

Using

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle, \quad (2.39)$$

we obtain

$$\langle Z \rangle = (\alpha^* \langle 0| + \beta^* \langle 1|) (\alpha |0\rangle - \beta |1\rangle) \quad (2.40)$$

$$= |\alpha|^2 - |\beta|^2. \quad (2.41)$$

Therefore, the expectation value of Z is the difference between the probabilities of measuring the qubit in the states $|0\rangle$ and $|1\rangle$:

$$\langle Z \rangle = P(0) - P(1). \quad (2.42)$$

2.2 Machine Learning

Machine learning, as its name suggests, is a way to make machines learn from data through specific algorithms.

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” [5]

To better understand this statement and the other underlying features of machine learning, we will carry out, step by step, a binary image classification task applied to a specific dataset. This running example will help introduce the main concepts of machine learning, tensor-network learning, and quantum machine learning that will be useful later in Chapters 3 and 4. The specific architectures and implementations used in the later experiments may, however, differ in some details.

2.2.1 The task T

Classification is typically considered a supervised learning task. In supervised learning, the model is trained using a labelled dataset, in which each input $\mathbf{x}^{(i)}$ is provided together with a corresponding target label $y^{(i)}$. The aim of classification is to learn a function

$$f : \mathbb{R}^n \rightarrow \{1, \dots, k\},$$

which assigns an input vector $\mathbf{x}$ to one of k possible classes [6]. The trained model should then be able to make predictions for previously unseen inputs. In some cases, the model may instead output a probability distribution over the possible classes. This differs from unsupervised learning, where the training data are not provided with target labels and the goal is instead to discover structure or patterns in the data.

From now on, we will focus on the classification of handwritten images of the digits 0 and 1 from the MNIST dataset. Therefore, we aim to learn a function

$$f : \mathbb{R}^n \rightarrow \{0, 1\},$$

where the input $\mathbf{x}$ is formed from the pixel values of the image. This corresponds to learning a decision function that partitions the feature space into regions associated with the two classes.

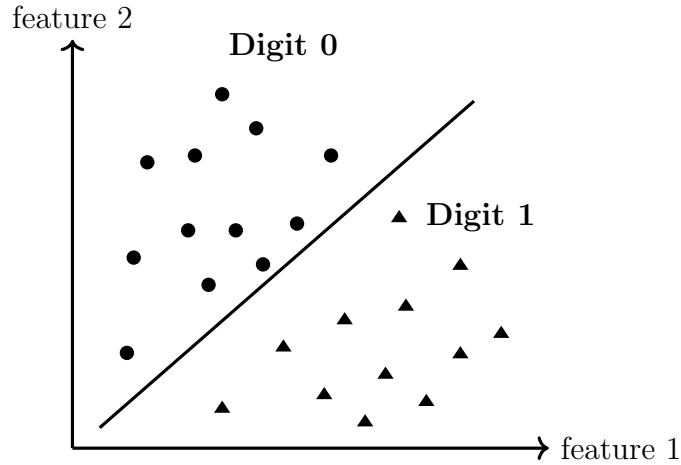


Figure 2.3: Conceptual illustration of binary classification in the feature space. Each point represents an image of a handwritten digit (0 or 1) from the MNIST dataset. The classifier learns a decision boundary that separates the two classes. For visualisation purposes, only two features are shown, whereas the original MNIST images contain 784 pixel features.

Remark: We call a feature an individual measurable property or characteristic used as input to a model. In our case, each pixel is a feature. In the present example, each pixel intensity constitutes one feature.

Note: We will not specify any model architecture here, e.g., neural networks. We will rather keep the discussion very general through the example.

2.2.2 Data Preparation and Dataset Partitioning

The MNIST database is commonly used as a benchmark for image recognition. The database contains 70,000 grayscale images of handwritten digits. These are stored as 28×28 pixel matrices. Pixel values are typically represented by integers between 0 and 255, where 0 denotes a black pixel and 255 denotes a white pixel. Thus, each image can be interpreted as a collection of 784 numerical features, one for each pixel. Digits go from 0 to 9, each associated with its particular digit. [7]

Only a part of this database (only digits 0 and 1) will be considered for our task of binary classification.

Each grayscale image corresponds to the vector $\mathbf{x}$ and has a label $y \in \{0, 1\}$ which is the class it belongs to. Hence, the entire dataset can be written as

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N \quad (2.43)$$

where N is the number of labeled data points.

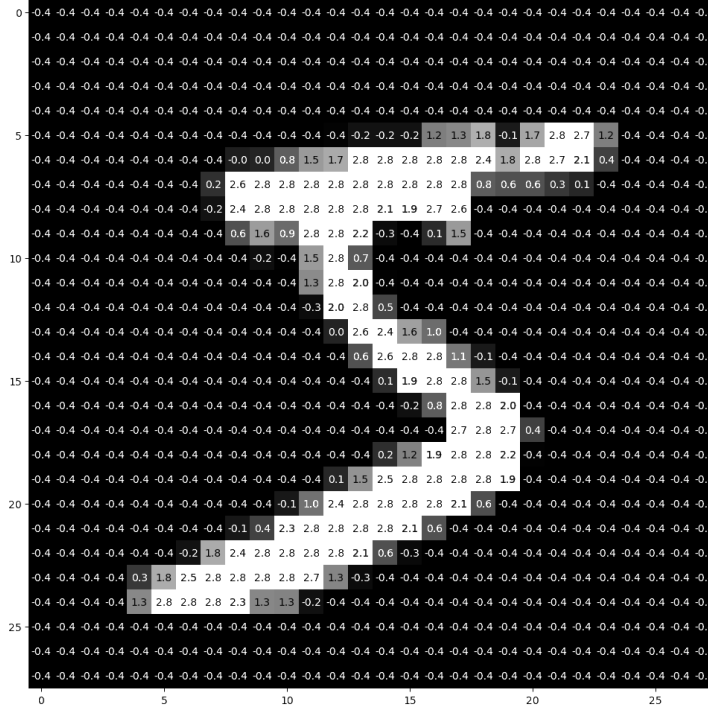


Figure 2.4: Illustration of the MNIST data used for binary classification. Each 28×28 grayscale image of a handwritten digit is represented as a 784-dimensional feature vector and assigned a label $y \in \{0, 1\}$.

Before passing our data to the learning algorithm, it needs to be preprocessed and partitioned. For MNIST, the pixel intensities can be rescaled from the original range $[0, 255]$ to $[0, 1]$. Such preprocessing can improve numerical stability and facilitate the optimisation of many machine-learning models.

For a good measure of performance, the dataset is partitioned into three disjoint subsets: a training set, a validation set, and a test set.

The training data are used to optimise the model parameters, while the validation data are used to assess if our model is learning properly, i.e., good generalisation, [6, Ch. 5]. See Section 2.2.5. Finally, the test set is reserved exclusively for the final evaluation of the trained model and provides an estimate of its predictive performance.

This partitioning strategy allows proper learning while providing an objective assessment of its predictive capability.

2.2.3 Feature mapping

Now that our data is ready, we should do one more thing before feeding it to our learning algorithm.

We have said earlier that our learning algorithm, without specifying what it is for now, is trying to learn a decision function that partitions the feature space into two regions. However, depending on the structure of the data, finding such a partition may be difficult,

and the decision boundary required to separate the classes may be highly non-linear. To simplify our task, we map the input

$$\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$$

into a new representation

$$\Phi(\mathbf{x}),$$

where the data are represented by a larger collection of features. By embedding the input into a higher-dimensional feature space, this richer representation allows the classifier to capture higher-order combinations of the input variables, which can make complex class structures easier to separate. In other words, although the classes may not be easily separable in the original input space, they may become easier to separate after the feature mapping. This principle is closely related to kernel methods, as described in Chapter [8, ch. 6]. This feature mapping will also be important because it allows a classifier that is linear in the transformed feature space to learn nonlinear decision boundaries in the original input space [9].

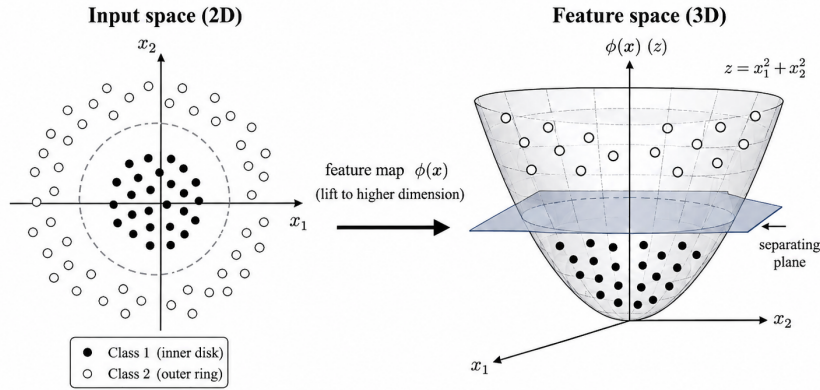


Figure 2.5: Illustration of a nonlinear feature map. In the original two-dimensional input space, the inner and outer classes are not linearly separable. After mapping the data into a higher-dimensional feature space, the two classes can be separated by a linear decision plane.

2.2.4 Training algorithm

Before continuing, we do a recap of some terminology.

- **Input:** $\mathbf{x}_i$ denotes the feature vector for a single training example i of the dataset D . Each individual feature is a scalar, for example x_j^i , where j indexes the feature. The full collection of input examples of D is written as the matrix $\mathbf{X}$.
- **Label:** y_i denotes the true target value for a single training example i . The vector containing the true target values for all examples is written as $\mathbf{y}$.

- **Prediction:** $\hat{y}_i$ denotes the predicted value for a single training example i , produced by the model. The vector containing the predictions for all examples is written as $\hat{\mathbf{y}}$.
- **Loss:** A numerical measure of how different the model's prediction is from the corresponding true label. For example, a loss may compare $\hat{y}_i$ with y_i for a single example, or $\hat{\mathbf{y}}$ with $\mathbf{y}$ over multiple examples.
- **Batch:** A subset of the training data processed together before the model parameters are updated.
- **Epoch:** One complete pass through the entire training dataset.

As discussed previously, the objective is to learn a function that predicts the correct label for a given input. To be able to produce this function f , we construct a model that depends on a set of parameters $\boldsymbol{\theta}$. The model can therefore be written as

$$\mathbf{y} = \mathbf{f}(\mathbf{X}; \boldsymbol{\theta}), \quad (2.44)$$

We can think of the model as a black box that receives the input $\mathbf{x}$ and produces an output that depends on the current values of its parameters. By changing the parameters, we also change the function implemented by the model.

Example: If we consider the simple linear model

$$\mathbf{f}(\mathbf{X}; \boldsymbol{\theta}) = \mathbf{w}^T \mathbf{X} + \mathbf{b}, \quad (2.45)$$

where

$$\boldsymbol{\theta} = \{\mathbf{w}, \mathbf{b}\}.$$

Different values of $\mathbf{w}$ and $\mathbf{b}$ produce different functions. The objective of the learning algorithm is therefore to find the parameter values that best describe the relationship between the input data and their labels.

Loss Function

For the model to improve, it needs to measure how far its prediction $\hat{y}$ is from the true output (label) y . Given an input $\mathbf{x}^{(i)}$, the model produces a prediction

$$\hat{y}^{(i)} = f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \quad (2.46)$$

which is then compared with the true label $y^{(i)}$. The discrepancy between the prediction and the true output is measured by a *loss function*,

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}). \quad (2.47)$$

A small loss indicates that the predictions agree well with the true outputs, whereas a large loss indicates poor agreement.

Several loss functions exist depending on the type of learning problem. Some commonly used loss functions [8, Ch. 1] are:

- **Mean Squared Error (MSE)** (regression)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2. \quad (2.48)$$

- **Mean Absolute Error (MAE)** (regression)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}|. \quad (2.49)$$

Here, N is the number of samples, $y^{(i)}$ is the true output associated with the i -th sample, and $\hat{y}^{(i)}$ is the corresponding model prediction.

In our example of binary image classification, we use the cross-entropy loss [8, Ch. 4]. For a given input, the model produces two logits,

$$\mathbf{z} = [z_0 \ z_1], \quad (2.50)$$

where the subscript refers to the corresponding class, either 0 or 1.

These logits are converted into probabilities using the softmax function,

$$p_i = \frac{e^{z_i}}{\sum_{j=0}^1 e^{z_j}}, \quad i \in \{0, 1\}. \quad (2.51)$$

The resulting probability vector is

$$\mathbf{p} = [p_0 \ p_1], \quad (2.52)$$

with

$$0 \leq p_i \leq 1, \quad p_0 + p_1 = 1. \quad (2.53)$$

For a true class label $y \in \{0, 1\}$, the cross-entropy loss is

$$\mathcal{L}_{\text{CE}} = -\log(p_y) = -\log\left(\frac{e^{z_y}}{e^{z_0} + e^{z_1}}\right). \quad (2.54)$$

The cross-entropy loss therefore depends only on the probability assigned to the correct class. If the model assigns a high probability to the correct class, the loss is small. Conversely, if it assigns a low probability to the correct class, the loss becomes large.

Example. Suppose the model produces the logits

$$\mathbf{z} = [2.3 \ 0.7] . \quad (2.55)$$

Applying the softmax function gives approximately

$$\mathbf{p} \approx [0.83 \ 0.17] . \quad (2.56)$$

If the true label is class 0, then the loss is

$$-\log(0.83) \approx 0.19, \quad (2.57)$$

which is small because the model assigns a high probability to the correct class. If instead the true label is class 1, then the loss is

$$-\log(0.17) \approx 1.77, \quad (2.58)$$

which is much larger because the model assigns a low probability to the correct class.

Remark. The logits depend on both the input and the parameters of the model. For the i -th sample, we can therefore write

$$\mathbf{z}^{(i)} = \mathbf{z}(\mathbf{x}^{(i)}; \boldsymbol{\theta}) . \quad (2.59)$$

Consequently, the loss for the i -th sample can be written as

$$\mathcal{L}^{(i)} = \mathcal{L}(\mathbf{z}(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}) . \quad (2.60)$$

Gradient descent

Now that we know how to calculate the loss, we want to adjust the parameters so that the loss is as small as possible. In other words, we want to find the parameters for which the loss is minimised.

Since we have $\mathcal{L} = \mathcal{L}(\boldsymbol{\theta})$, we can calculate the gradient of the loss function with respect to the parameters:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1} \\ \frac{\partial \mathcal{L}}{\partial \theta_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_P} \end{bmatrix} . \quad (2.61)$$

This gives us the direction of steepest ascent of the loss function. Therefore, we change the parameters by taking a small step in the opposite direction of the gradient. This optimisation procedure is called gradient descent. The parameter update is given by

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k), \quad (2.62)$$

where $\eta > 0$ is called the learning rate. The learning rate determines the size of the update at each iteration. If it is too large, the algorithm may overshoot the minimum, whereas if it is too small, learning becomes very slow.

The gradients are computed efficiently using an algorithm called backpropagation. Backpropagation applies the chain rule to compute the gradient of the loss with respect to every parameter of the model.

These gradients are then used to do gradient descent to update the parameters according to the previous update rule. In other words, backpropagation computes the gradients, while gradient descent uses these gradients to improve the model.

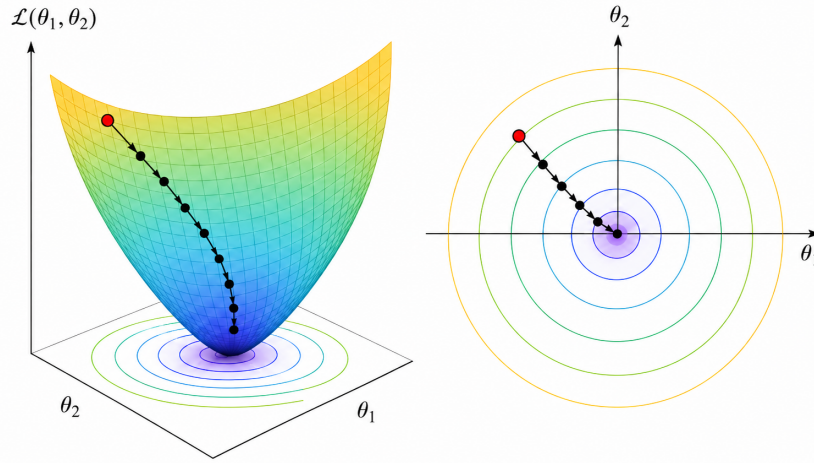


Figure 2.6: Illustration of gradient descent in parameter space. The left panel shows the optimisation trajectory on the loss surface $\mathcal{L}(\theta_1, \theta_2)$, while the right panel shows the same trajectory projected onto the corresponding contour plot.

During training, the model repeatedly processes one batch of data at a time [6, Ch. 8]. For each batch, it computes its predictions, evaluates the loss, calculates the gradients using backpropagation, updates its parameters using gradient descent, and repeats this process for every batch in every epoch until the model converges. Gradient descent provides the basic principle underlying gradient-based optimisation. In this work, however, the parameters will be updated using the Adam optimiser, which adapts the effective learning rate of each parameter using running estimates of the first and second moments of the gradients [10].

2.2.5 Generalisation and Model Complexity

The Performance measure P

Our model can now learn, but we still need to define a performance measure. To have a good measure, we should clearly know what we want our model to do. First, we would like the model to classify most of the data we give to it correctly during training. We would also like our model to correctly classify most of the unseen data we give it. Unseen data is the data that has not been used at all during training. A natural measure of both of these is the accuracy, defined as

$$\text{Accuracy} = \frac{\text{Number of correctly classified examples}}{\text{Total number of examples}}. \quad (2.63)$$

Another important measure is the loss itself, which shows how confident our model is in its predictions [6, Sec. 5.5].

These measurements make even more sense if we remember that we partitioned our data earlier into three sets [8, Sec. 1.3]:

- **Training set:** used to learn the model parameters.
- **Validation set:** used to monitor the model during training and to select hyperparameters.
- **Test set:** used only after training to evaluate the final performance of the model on unseen data.

By measuring these metrics on the different sets during the learning process, we are able to know if our model is learning correctly or not.

Overfitting and Underfitting

As stated, we want our model to have high accuracy not only on the training data but also on unseen data. To do so, it needs to learn the underlying pattern of the dataset, not just memorise each data point in the training dataset. This ability is known as *generalisation* [8, Sec. 1.3]. A model generalises well when its training and validation (or test) performance are both high and reasonably close.

A measure of generalisation is the generalisation gap

$$\Delta_{\text{gen}} = \mathcal{L}_{\text{val}} - \mathcal{L}_{\text{train}}, \quad (2.64)$$

or

$$\Delta_{\text{acc}} = \text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{val}}. \quad (2.65)$$

It is better to have a small generalisation gap, although it must be considered together with the absolute performance of the model [6, Ch. 5]. There are 2 syndromes of a model generalising badly: overfitting and underfitting.

Underfitting Underfitting is when the model is unable to correctly learn the patterns in the data.

It arises when

- the model is insufficiently expressive;
- training is stopped too early;
- the learning rate prevents effective optimisation;
- the chosen feature representation does not expose the relevant class structure. [6, Ch. 5]

Overfitting Overfitting occurs when a model fits the training data very well but performs substantially worse on unseen data resulting in a big generalisation gap. The model starts to learn the noise in the data and very specific details in the training data. Therefore, when it is given unseen data, it performs poorly [6, Ch. 5].

Different strategies exist to prevent overfitting, such as

- reducing model complexity;
- using more training data;
- applying regularisation;
- stopping training when the validation performance no longer improves (early stopping) [11].

Overfitting and underfitting are deeply related to the ability of the model to learn the complex structure of the dataset. This ability is known as *expressivity*. Expressivity refers to the range of functions that a model is capable of representing [12]. Insufficient expressivity leads to underfitting, whereas excessive effective complexity increases the risk of overfitting. The number of trainable parameters is one factor that determines the expressivity of a model, but the architecture, parameterisation, optimisation procedure, and structure imposed on the model also affect which functions can be learned in practice.

Hyperparameters

A distinction must be made between model parameters and hyperparameters. Model parameters are learned during training. In this work, these include the entries of the trainable tensors or the rotation angles in the parameterised quantum circuit.

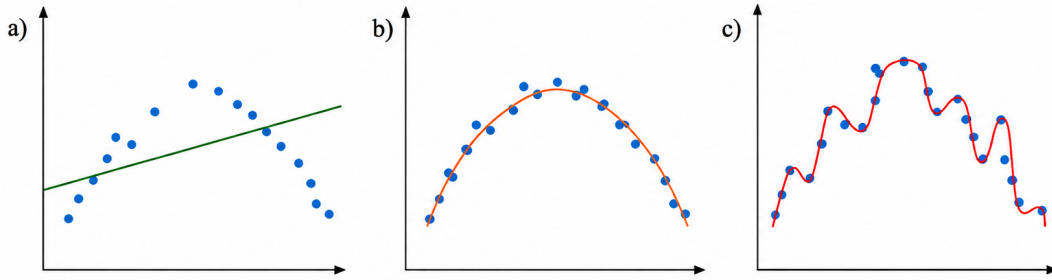


Figure 2.7: Illustration of model complexity: **(a)** underfitting, where the model is too simple to capture the underlying pattern; **(b)** an appropriate fit, where the model captures the general trend; and **(c)** overfitting, where the model follows noise and small fluctuations in the training data.

Hyperparameters are chosen before or during model development and are not directly learned by gradient descent [6, Ch. 11]. Some examples include the learning rate, batch size, and number of epochs. Hyperparameters influence both optimisation and generalisation.

For example, a learning rate that is too large may cause unstable training, while a learning rate that is too small may lead to slow convergence. Hyperparameters are therefore normally selected by comparing model performance on the validation set. Once the hyperparameters have been fixed, the test set is used to report the final performance [8, Sec. 1.3].

2.3 Tensor Network

Tensor networks, as we know them today, first appeared when Penrose introduced his diagrammatic representation of tensor contractions in 1971 [13]. The idea was then further developed by White in 1992 with the Density Matrix Renormalization Group (DMRG) algorithm, which efficiently computes the low-energy properties of one-dimensional quantum lattice systems by using reduced density matrices to optimise basis truncation [14]. This concept was later reformulated in terms of Matrix Product States (MPS) by Östlund and Rommer in 1995 [15], and was itself generalised into more complex structures such as Projected Entangled Pair States (PEPS), Tree Tensor Networks (TTNs), and the Multi-scale Entanglement Renormalization Ansatz (MERA), giving rise to the modern tensor network framework. However, it was only in 2008 that Verstraete, Murg, and Cirac

provided a unified formalism for tensor networks [16].

2.3.1 Matrix Product States (MPS)

Matrix Product States (MPS) are tensor networks that can be viewed as the factorisation of a tensor with N indices into a product of lower-order tensors [17, 18]. For example,

$$T_{s_1 s_2 s_3 s_4 s_5 s_6} = \sum_{\{\alpha\}} A_{s_1, \alpha_1}^{[1]} A_{s_2, \alpha_1 \alpha_2}^{[2]} A_{s_3, \alpha_2 \alpha_3}^{[3]} A_{s_4, \alpha_3 \alpha_4}^{[4]} A_{s_5, \alpha_4 \alpha_5}^{[5]} A_{s_6, \alpha_5}^{[6]}. \quad (2.66)$$

Here, the indices $s_1, \dots, s_6$ are the physical indices, while $\alpha_1, \dots, \alpha_5$ are the bond indices connecting neighbouring tensors. The tensors at the ends have two indices, while the tensors in the middle have three indices. Alternatively, all tensors can be treated as three-index tensors by taking the boundary bond dimensions to be one.

Or, in tensor diagram notation, as shown in Figure 2.8.

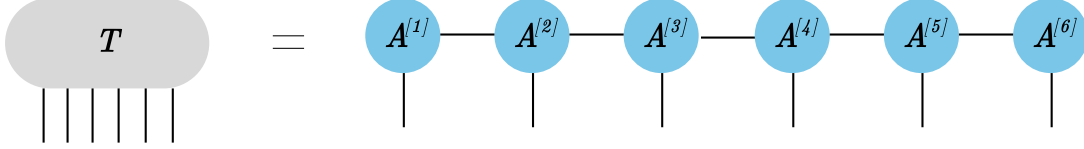


Figure 2.8: Tensor-network representation of a six-site matrix product state. The original tensor T is factorised into a one-dimensional chain of lower-order tensors $A^{[1]}, \dots, A^{[6]}$, connected by internal bond indices.

Bond dimension The bond dimension is the dimension of the bond indices connecting neighbouring tensors in the MPS. If the dimensions of the bond indices are denoted by χ_k , then the bond dimension of the MPS is usually defined as

$$\chi = \max_k \chi_k.$$

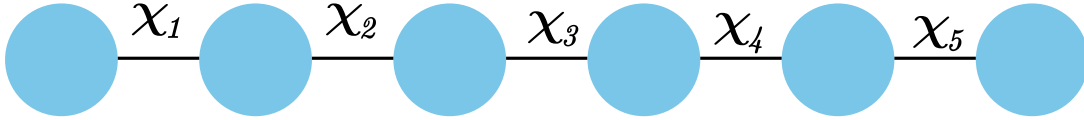


Figure 2.9: Illustration of the bond dimensions of a six-site MPS. The bond index connecting tensors k and $k + 1$ has dimension χ_k , and the overall bond dimension is defined as $\chi = \max_k \chi_k$.

This quantity will be important when discussing the expressivity and computational cost of MPS in machine learning [9].

Any tensor $T_{s_1 s_2 \dots s_N}$ can be represented exactly as an MPS. If each physical index has dimension d , then the bond dimension across the k -th bipartition satisfies

$$\chi_k \leq d^{\min(k, N-k)}.$$

Therefore, the maximum bond dimension required for an exact representation satisfies

$$\chi \leq d^{\lfloor N/2 \rfloor},$$

where the upper bound is attained for generic tensors [19, 20].

MPS with Output Index

In machine-learning applications, it is often convenient to leave one or more physical indices of an MPS uncontracted. The resulting tensor has multiple output indices while retaining the same one-dimensional chain structure and bond-index connectivity as a standard MPS.

For example,

$$Y_t = \sum_{\{\beta\}} A_{\beta_1}^1 A_{\beta_1 \beta_2}^2 A_{\beta_2 \beta_3}^3 A_{\beta_3 \beta_4}^4 A_{\beta_4 \beta_5}^5 A_{\beta_5, t}^6. \quad (2.67)$$

Here, the index t is the output physical index, while $\beta_1, \dots, \beta_{N-1}$ are the bond indices connecting neighbouring tensors. Tensors that are associated with an output carry one physical index in addition to their bond indices, whereas the remaining tensors contain only bond indices. The tensors at the ends have fewer bond indices than those in the middle; equivalently, all tensors may be regarded as three-index tensors by taking the boundary bond dimensions to be one.

Or, in tensor diagram notation, as shown in Figure 2.10.

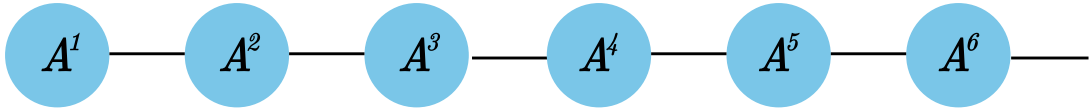


Figure 2.10: Tensor-network representation of an MPS with an uncontracted output index t at the final tensor. The horizontal lines represent the bond indices connecting neighbouring tensors.

2.3.2 Tensor Networks for Machine Learning

This section was greatly inspired by [21].

Now that we have some basic understanding of Matrix Product States (MPS), we can use closely related tensor-network structures for our machine-learning task. In the classification setting considered here, the tensor network has physical indices associated with the

input data as well as an output physical index associated with the class label. According to the convention adopted here, a tensor network containing only input or only output physical legs is referred to as an MPS, whereas a tensor network containing both input and output physical legs is referred to as a Matrix Product Operator (MPO). Therefore, the classifier introduced below is represented as an MPO.

The MPO is parameterised by a set of tensors

$$\theta = \{A^{(1)}, A^{(2)}, \dots, A^{(N)}\},$$

whose entries are the trainable parameters learned during training.

We now follow the same general machine-learning procedure introduced previously, but make each step specific to the tensor-network model.

Data preparation The data preparation remains the same as explained in Section 2.2.2.

Data encoding

For the reasons discussed previously, we use a feature map before passing the data into the model. Each pixel of the image is encoded into its own two-dimensional feature space according to the local feature map [9, 21, 23],

$$\Phi(p) = \begin{bmatrix} \cos\left(\frac{\pi}{2}p\right) \\ \sin\left(\frac{\pi}{2}p\right) \end{bmatrix} \quad (2.68) \text{ where } p \in [0, 1] \text{ is the pixel value normalised to the unit interval.}$$

The encoded image is then represented as the tensor product

$$\Phi(\mathbf{x}) = \Phi(p_1) \otimes \Phi(p_2) \otimes \dots \otimes \Phi(p_N). \quad (2.69)$$

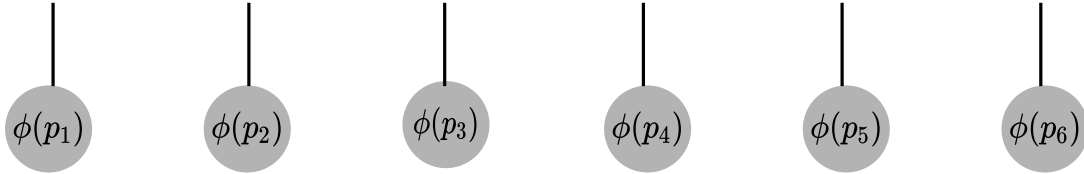


Figure 2.11: Product feature map used to encode an image for the tensor-network classifier. Each pixel p_i is mapped independently to the two-dimensional feature vector $\Phi(p_i)$, and the complete encoded input is the tensor product $\Phi(\mathbf{x}) = \Phi(p_1) \otimes \dots \otimes \Phi(p_N)$.

Learning

The tensor-network classifier represents a tensor with input physical indices $i_1, \dots, i_N$ and an output physical index l . Since both input and output physical legs are present, this tensor network is an MPO under the convention introduced above.

$$T_{i_1 i_2 \dots i_N, l} = \sum_{\alpha_1, \dots, \alpha_{N-1}} A_{i_1 \alpha_1}^{[1]} A_{i_2 \alpha_1 \alpha_2}^{[2]} A_{i_3 \alpha_2 \alpha_3}^{[3]} \dots A_{i_N \alpha_{N-1}, l}^{[N]}. \quad (2.70)$$

where $\alpha_1, \dots, \alpha_{N-1}$ are the internal bond indices. The ranges of the bond indices α_k are determined by the bond dimension, which is a hyperparameter of the model. Increasing the bond dimension increases the size of the local tensors and generally allows the tensor network to represent a larger class of functions [17, 18]. The entries of the tensors $A^{(k)}$ are the trainable parameters learned during training.

As mentioned in Section 2.2.1, we want the model to map each handwritten image to its corresponding digit from the set $\{0, 1\}$. Following [9], this is achieved by contracting the encoded image with the input physical indices of the variational MPO. The output index l is left uncontracted, giving

$$z_l = \sum_{i_1, i_2, \dots, i_N} T_{i_1 i_2 \dots i_N, l} \Phi(p_1)_{i_1} \Phi(p_2)_{i_2} \cdots \Phi(p_N)_{i_N}, \quad (2.71)$$

where $l \in \{0, 1\}$ labels the output class.

For binary classification, the contraction therefore produces the two logits

$$\mathbf{z} = \begin{bmatrix} z_0 \\ z_1 \end{bmatrix}.$$

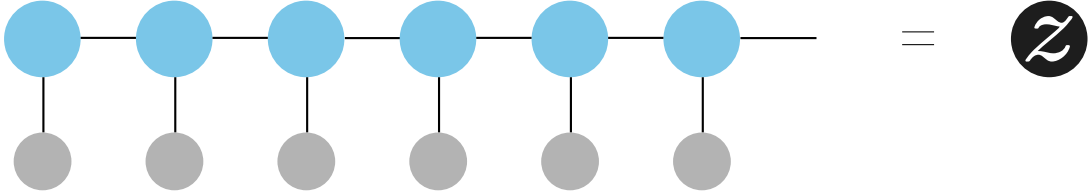


Figure 2.12: Illustration of tensor-network classification. The local feature vectors are contracted with the input physical indices of the MPO, while the output physical index remains uncontracted. For binary classification, this output index produces the two class logits z_0 and z_1 .

These logits are then passed to the cross-entropy loss introduced in Section 2.2.4,

$$\mathcal{L}(\mathbf{z}, y) = -\log \left(\frac{e^{z_y}}{e^{z_0} + e^{z_1}} \right),$$

where y is the true class label.

The parameters are then updated using gradient-based optimisation through backpropagation, as explained in Section 2.2.4. In the implementation, the gradients are computed automatically and the parameters are updated using the Adam optimiser [10].

In many physics applications, sweeping algorithms such as the Density Matrix Renormalization Group (DMRG) are preferred because they can be more efficient for tensor-network optimisation [14, 18]. Nevertheless, gradient-based optimisation can also be used for tensor-network models. Its simplicity and compatibility with modern machine-learning frameworks make it a convenient choice for supervised-learning applications.

In the remainder of this dissertation, we will nevertheless use the term MPS when referring to the underlying one-dimensional tensor-network parameterisation used for classical pretraining, while MPO refers to the full input-output tensor represented by this network.

2.4 Quantum Machine Learning

We have seen that tensor networks, and in particular Matrix Product Operator, can be used as machine learning model architectures. We will now see that quantum circuits can also play this role, giving rise to the field of quantum machine learning (QML). Quantum machine learning combines ideas from quantum computing and machine learning to develop learning algorithms that can be implemented on quantum hardware [24, 25]. Since its introduction, many quantum machine learning models have been proposed [25, 31].

These include quantum kernel methods [26] and [38], quantum neural networks [39], quantum convolutional neural networks [28], and variational quantum circuits (VQCs) [31]. In this work, we focus on variational quantum circuits because, similarly to an MPS, they are parameterised models whose parameters are learned from data through optimisation.

Parameterised quantum circuits can exploit superposition and entanglement to represent states in an exponentially large Hilbert space. Properly designed circuits may therefore provide highly expressive models using a comparatively compact set of trainable parameters [30].

QML is especially natural when the input data already comes from a quantum system, such as molecules, quantum materials, or particle-physics experiments. In this case, a quantum computer can process these states directly, avoiding the potentially expensive conversion or classical simulation of quantum data [31].

As before, we consider the binary classification of handwritten images. The objective is therefore to learn a function that allows us to perform our binary classification task using a variational quantum circuit as the model architecture.

2.4.1 Variational Quantum Models

A variational quantum circuit is a parameterised quantum circuit made of layers of single-qubit rotation gates and entangling gates. The rotation angles form the trainable parameters of the model, while the entangling gates allow correlations to develop between different qubits [29, 32].

Data preparation

As in the previous section, the data preparation remains unchanged. The dataset is first preprocessed, normalised, and partitioned into training, validation, and test sets.

Data encoding

Unlike classical machine learning models, quantum circuits operate on quantum states rather than classical vectors. Therefore, before the data can be processed by the circuit, each classical input must first be encoded into a quantum state.

As explained in Section 2.1.3, we use angle encoding. Given an input vector

$$x = (x_1, \dots, x_N),$$

each feature is encoded as the rotation angle of a single-qubit rotation gate,

$$|\psi(x)\rangle = \bigotimes_{i=1}^N R_y(\pi x_i) |0\rangle, \quad (2.72)$$

where

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \quad (2.73)$$

The encoded quantum state is then passed through a parameterised quantum circuit composed of single-qubit rotation gates and entangling gates. The rotation angles of the variational gates are the trainable parameters of the model,

$$\theta = \{\theta_1, \theta_2, \dots, \theta_P\}.$$

Just as the entries of the MPO tensors were learned during training, here the parameters of the quantum gates are learned from the training data. After the data have propagated through the circuit, expectation values are measured to produce the model outputs. For our binary classification task, the circuit produces two logits,

$$z = \begin{bmatrix} z_0 \\ z_1 \end{bmatrix},$$

which are passed directly to the same cross-entropy loss introduced previously,

$$\mathcal{L} = -\log \left(\frac{e^{z_y}}{e^{z_0} + e^{z_1}} \right).$$

The parameters are then updated using gradient-based optimisation, as described in Section 2.2.4. In our simulations, the gradients are computed automatically through backpropagation and the parameters are updated using the Adam optimiser. On real quantum hardware, however, gradients must instead be estimated from quantum-circuit measurements.

2.4.2 Challenges and Limitations

A barren plateau is a region in the parameter space of a VQC where the loss landscape becomes extremely flat. More precisely, under conditions that give rise to barren plateaus, the expectation value of the gradient is typically zero,

$$\mathbb{E}_{\theta} \left[\frac{\partial \mathcal{L}}{\partial \theta_i} \right] = 0, \quad (2.74)$$

while its variance can decrease exponentially with the number of qubits. A characteristic scaling is

$$\text{Var}_{\boldsymbol{\theta}} \left[\frac{\partial \mathcal{L}}{\partial \theta_i} \right] \sim \mathcal{O}(2^{-n}) \quad (2.75)$$

[33].

The corresponding root-mean-square scale of the gradient is therefore

$$\sqrt{\text{Var}_{\boldsymbol{\theta}} \left[\frac{\partial \mathcal{L}}{\partial \theta_i} \right]} \in \mathcal{O}(2^{-n/2}). \quad (2.76)$$

Thus, as the number of qubits increases, the gradients become increasingly concentrated around zero.

The main difficulty is not only that the gradients are small, but that they become increasingly difficult to estimate reliably from a finite number of quantum measurements. On quantum hardware, expectation values and gradients are estimated statistically from repeated circuit executions. If the signal that we want to resolve decreases exponentially with the number of qubits, an increasing number of measurements is required to distinguish this signal from the statistical fluctuations of the estimator.

A simple analogy is to consider choosing uniformly at random one integer from the set

$$\{1, 2, \dots, 2^n\}. \quad (2.77)$$

Each possible value occurs with probability 2^{-n} . If only a small number of samples is taken compared with 2^n , most possible values will never be observed. In the same way, exponentially small features of a probability distribution can require an exponentially large number of samples before they can be resolved with statistical confidence.

For barren plateaus, this means that even though the gradient is in principle well defined, obtaining a sufficiently accurate estimate of its direction can become very expensive. If the statistical uncertainty in the gradient estimate is comparable to or larger than the gradient itself, the optimiser cannot reliably determine in which direction the parameters should be updated. The parameter update

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k) \quad (2.78)$$

therefore becomes difficult to perform reliably, since the measured gradient may be dominated by statistical uncertainty.

Barren plateaus can be understood as a particular case of exponential concentration. In sufficiently high-dimensional spaces, suitably regular functions may become strongly concentrated around their expectation values. For a function $f(\boldsymbol{\theta})$, a concentration bound may take the form [33]

$$\Pr(|f(\boldsymbol{\theta}) - \mathbb{E}_{\boldsymbol{\theta}}[f(\boldsymbol{\theta})]| \geq \varepsilon) \leq Ae^{-Bd\varepsilon^2}, \quad (2.79)$$

where A and B are positive constants and d is the dimension of the space. In the case of barren plateaus, this exponential concentration appears in the gradients of the cost

function, whose distribution becomes increasingly concentrated around zero as the size of the quantum system increases.

In parameterised quantum circuits that behave sufficiently like random high-dimensional systems, this concentration can cause the cost function to vary very little over large regions of parameter space. Its derivatives are then strongly concentrated around zero. Consequently, resolving the gradient with sufficient statistical confidence can require a very large number of circuit measurements, making the optimisation of the VQC increasingly difficult.

Computational Cost of VQCs

This high cost is mainly because quantum measurements are probabilistic, as discussed in Section 2.1.3. Therefore, a circuit must generally be executed repeatedly, using what are called shots, in order to estimate an expectation value.

For example, consider measuring the Pauli- Z observable on a single qubit. Each measurement returns either $+1$ or -1 . If the probability of obtaining $+1$ is p , then

$$\langle Z \rangle = p - (1 - p) = 2p - 1. \quad (2.80)$$

After S independent measurements, we can estimate p from the observed frequency $\hat{p}$. Since each measurement is a Bernoulli random variable, Hoeffding's inequality gives [40]

$$\Pr(|\hat{p} - p| \geq \epsilon) \leq 2e^{-2S\epsilon^2}. \quad (2.81)$$

Therefore, to estimate p to an additive error ϵ with failure probability at most δ , it is sufficient to take

$$S \geq \frac{1}{2\epsilon^2} \ln \left(\frac{2}{\delta} \right). \quad (2.82)$$

For fixed confidence level δ , the number of shots therefore scales as

$$S = \mathcal{O} \left(\frac{1}{\epsilon^2} \right). \quad (2.83)$$

The same scaling applies to estimating bounded expectation values from repeated measurements. Thus, higher accuracy requires many more hardware repetitions [34]. For example, reducing the statistical error by a factor of 10 may require approximately 100 times more shots. This makes the estimation of expectation values one of the main computational costs of variational quantum algorithms.

Chapter 3

Dense $SO(n)$ parameterisation

As seen in Section 2.4.2, VQCs have several problems. These problems include barren plateaus and the high cost of running algorithms on quantum computers. Here, we propose a new method that aims to reduce the amount of VQC optimisation required to perform a given task. The idea is to give the VQC a hot start through good parameter initialisation, so that fewer VQC optimisation steps may be required to reach a good solution.

We use tensor networks because their structure enables them to be easily converted into quantum circuits. This will be done by first pretraining a matrix product state (MPS) classically on the same task and then mapping it to a VQC.

Similar work has been done in [36] and [37]. However, these works use an approximate mapping of the tensor network and require intermediate decomposition algorithms, such as singular value decomposition (SVD).

Our approach is different because we directly parameterise the tensor network. This allows us to directly transfer the trained parameters from the MPS to the VQC, without requiring any intermediate decomposition step.

3.1 MPS to VQC

We will use a typical MPS-to-quantum-circuit mapping, as described in [37], but we will present the procedure in a more intuitive way. Only the main idea of the mapping will be discussed here, with particular emphasis on our contribution, namely the direct parameter transfer. A more detailed explanation of the complete mapping procedure is provided in Appendix A

Dense $SO(n)$ parameterisation. The parent matrices used to construct the MPS tensors are parameterised as special orthogonal matrices,

$$U(\boldsymbol{\theta}) \in SO(n),$$

where $U^T U = I$ and $\det(U) = 1$. We construct these matrices using a sequence of Givens rotations. Each Givens rotation acts as a two-dimensional rotation on a selected pair of

coordinates and is parameterised by a single angle θ_i . The complete matrix can therefore be written as

$$U(\boldsymbol{\theta}) = G_P(\theta_P) \cdots G_2(\theta_2)G_1(\theta_1), \quad (3.1)$$

where

$$P = \frac{n(n-1)}{2}$$

is the number of trainable parameters of a general matrix in $SO(n)$.

The MPS tensors are constructed from selected parts of these matrices, whereas the corresponding VQC gates use the complete matrices. Therefore, the same parameter vector $\boldsymbol{\theta}$ parameterises both models, allowing the parameters learned by the MPS to be transferred directly to the VQC. Figure 3.2 illustrates the main idea behind our MPS-to-VQC mapping.

Class 0	Class 0	Class 1	Class 1
0 1	1 0	0 0	1 1
0 1	1 0	1 1	0 0

Figure 3.1: The four 2×2 Bars-and-Stripes patterns used to construct the classification dataset. Vertical-bar patterns are assigned to class 0, while horizontal-stripe patterns are assigned to class 1.

3.2 Experiments

We first test the dense $SO(n)$ parameterisation on a binary classification task using the Bars-and-Stripes dataset. The dataset is composed of binary images containing either vertical bars or horizontal stripes. In the experiments presented here, we use 2×2 binary images, corresponding to four input sites. Vertical bars are assigned to class 0, while horizontal stripes are assigned to class 1.

The dataset is divided into 800 training samples and 200 validation samples. In these experiments, the validation samples are used to monitor the performance of the model. Throughout this chapter, the terms “validation set” and “test set” are used interchangeably to refer to this validation set. No separate held-out test set is used. A small amount of noise is also added to the images by randomly flipping each pixel with probability 0.05. This makes the classification task less trivial and allows us to test whether the models are able to learn the underlying Bars-and-Stripes structure rather than simply memorising perfect patterns.

The same input information is given to the MPS and the VQC. For each pixel $x \in [0, 1]$, we use a feature map similar to that described in Sections 2.3.2 and 2.4.1

$$\phi(x) = \begin{pmatrix} \cos\left(\frac{\pi x}{2}\right) \\ \sin\left(\frac{\pi x}{2}\right) \end{pmatrix}. \quad (3.2)$$

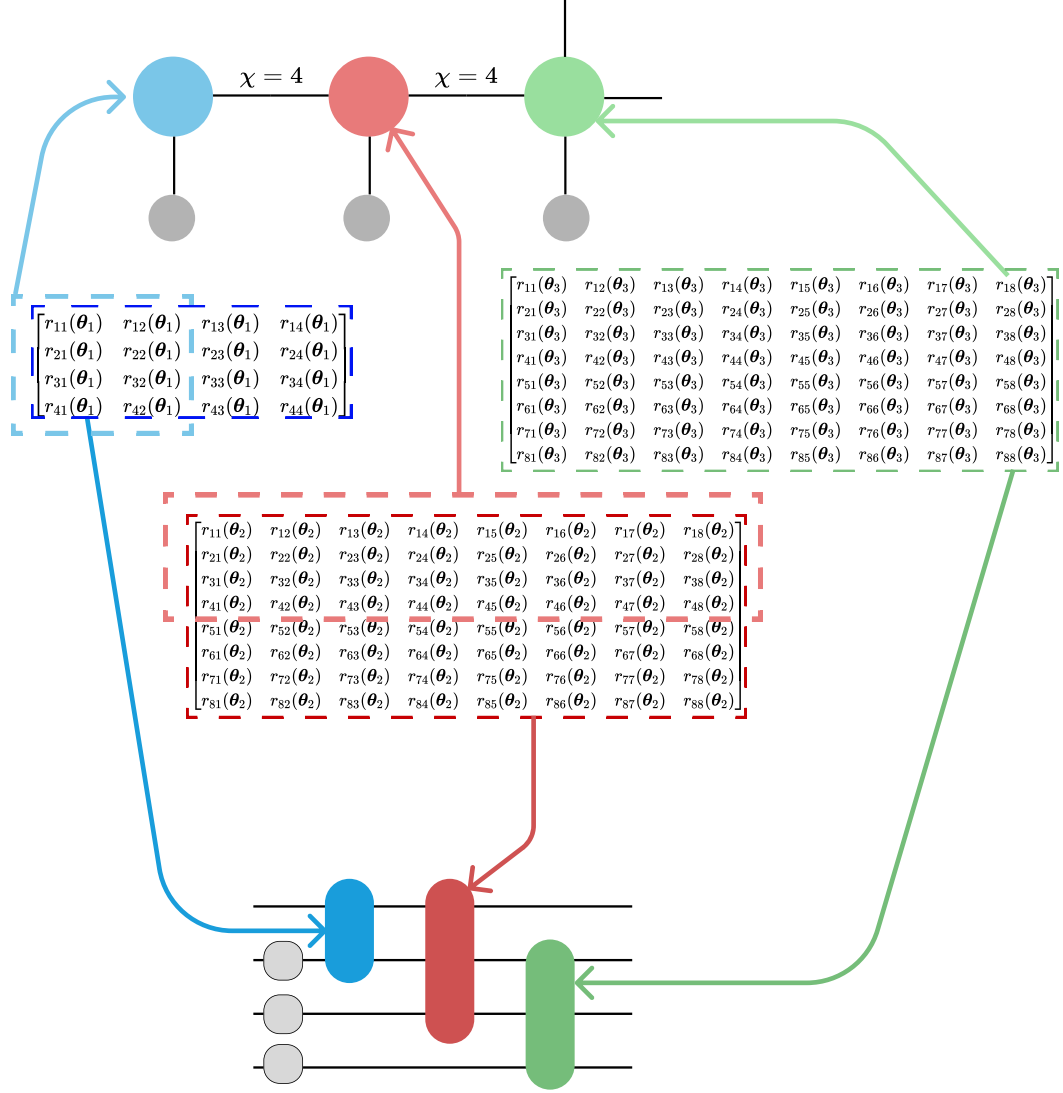


Figure 3.2: Illustration of the direct MPS-to-VQC mapping. The MPS tensors and the corresponding VQC gates are generated from the same parameterised orthogonal matrices. The MPS uses selected parts of these matrices, while the VQC uses the complete matrices as quantum gates. After classical MPS training, the optimised parameters are transferred directly to the corresponding VQC gates.

For the MPS, this vector is contracted directly with the physical index of the corresponding tensor. In the quantum circuit, the same state is prepared by applying an $R_y(\pi x)$ rotation to the corresponding input qubit. Therefore, the MPS and the VQC receive the same encoded input data. Both models are trained using the negative log-likelihood of the probability assigned to the correct class. For a training example with label $y \in \{0, 1\}$, the loss is

$$\mathcal{L} = -\log p_y, \quad (3.3)$$

where p_y is the probability assigned by the model to the correct class. The detailed construction of the MPS and VQC class probabilities is given in appendices A and B

All experiments are performed using classical simulation of the quantum circuits rather than real quantum hardware. Both the MPS and VQC are optimised using the Adam optimiser with a learning rate of 10^{-3} . The MPS is trained for 100 epochs with a batch size of 32, while the VQC is trained for 30 epochs with a batch size of 16. During VQC training, the circuit is simulated exactly without finite-shot sampling, and gradients are computed using backpropagation. Therefore, the results presented here demonstrate the effect of MPS pretraining on the VQC optimisation in simulation and do not directly measure the computational cost that would be obtained on real quantum hardware. To assess the robustness of the results to random initialisation, each experiment was repeated over 5 independent random seeds. The curves show the mean performance across these runs, and the error bars indicate one standard deviation. For visual clarity, error bars are shown only at selected epochs.

3.2.1 Effect of the bond dimension

The experiment is repeated for three different bond dimensions, $\chi = 2$, $\chi = 4$, and $\chi = 8$. The bond dimension controls the dimension of the internal bond space of the MPS and therefore affects the capacity of the model. A larger bond dimension allows the MPS to carry a richer internal representation as the input is contracted through the network.

In our parameterisation, changing χ also changes the size of the orthogonal matrices from which the MPS tensors and the corresponding VQC gates are generated. The first tensor is constructed from an $SO(\chi)$ matrix, while the intermediate transformations are constructed from $SO(2\chi)$ matrices. Increasing χ therefore increases both the expressivity of the MPS and the size of the corresponding transformations.

Since a general matrix in $SO(n)$ has

$$\frac{n(n-1)}{2} \quad (3.4)$$

independent parameters, increasing χ also substantially increases the number of trainable parameters in the shared MPS-VQC parameterisation. For the four-site problem considered here, the first transformation belongs to $SO(\chi)$, while the two intermediate transformations and the final transformation belong to $SO(2\chi)$. The total number of shared trainable parameters is therefore

$$P(\chi) = \frac{\chi(\chi-1)}{2} + 3\frac{(2\chi)(2\chi-1)}{2}. \quad (3.5)$$

This gives $P = 19, 90$, and 388 for $\chi = 2, 4$, and 8 , respectively. Thus, increasing the bond dimension gives the MPS substantially more freedom during the classical pretraining stage, while simultaneously increasing the size of the corresponding VQC parameterisation.

We therefore expect the bond dimension to affect how well the MPS can learn the classification task and, consequently, the quality of the parameters transferred to the VQC. However, increasing χ also increases the size of the model and the number of trainable parameters. By considering $\chi = 2, \chi = 4$, and $\chi = 8$, we can therefore investigate how the capacity of the MPS affects the usefulness of the pretrained VQC initialisation.

The purpose of this comparison is not only to determine which model reaches the highest final accuracy. The main goal is to determine whether MPS pretraining places the VQC at a better initial point in parameter space. For this reason, we compare both the training loss and the test accuracy during VQC training. In particular, epoch zero corresponds to the VQC immediately after initialisation, before any VQC optimisation has been performed. The performance at this point directly shows the effect of the classical MPS pretraining.

Results and discussion

Figure 3.3 shows that the effect of MPS pretraining depends on the bond dimension. In all three cases, the non-pretrained VQC is able to learn the classification task after optimisation. However, the initial point of the optimisation is affected by whether the parameters have first been trained using the MPS.

For $\chi = 2$, the non-pretrained VQC starts with a considerably lower training loss than the pretrained VQC. The pretrained VQC also initially performs worse in terms of test accuracy. After a small number of training epochs, however, both VQCs reach a similar region. This suggests that, for this small bond dimension, the MPS pretraining does not provide a clear advantage to the initialisation of the VQC.

The weaker hot-start behaviour for $\chi = 2$ is consistent with the more restricted parameterisation available at small bond dimension. As discussed above, the $\chi = 2$ model contains only 19 shared trainable parameters, compared with 90 for $\chi = 4$ and 388 for $\chi = 8$. The smaller model therefore has substantially fewer degrees of freedom with which to adapt the MPS representation during classical pretraining. The parameter vector obtained from this more restricted optimisation may consequently provide a less favourable initial point for the corresponding VQC.

Importantly, this does not mean that the parameter transfer is incomplete. All parameters learned by the MPS are transferred directly to the corresponding VQC gates. However, the MPS and VQC use the resulting transformations differently and therefore do not generally have identical loss landscapes.

The effect of pretraining becomes much clearer for $\chi = 4$. At epoch zero, the pretrained VQC already has a low training loss and a high test accuracy, while the non-pretrained VQC starts with a much larger loss and lower accuracy. The non-pretrained circuit is still able to reach a similar solution, but it requires several optimisation steps to do so. In contrast, a large part of the optimisation of the pretrained VQC has already been carried

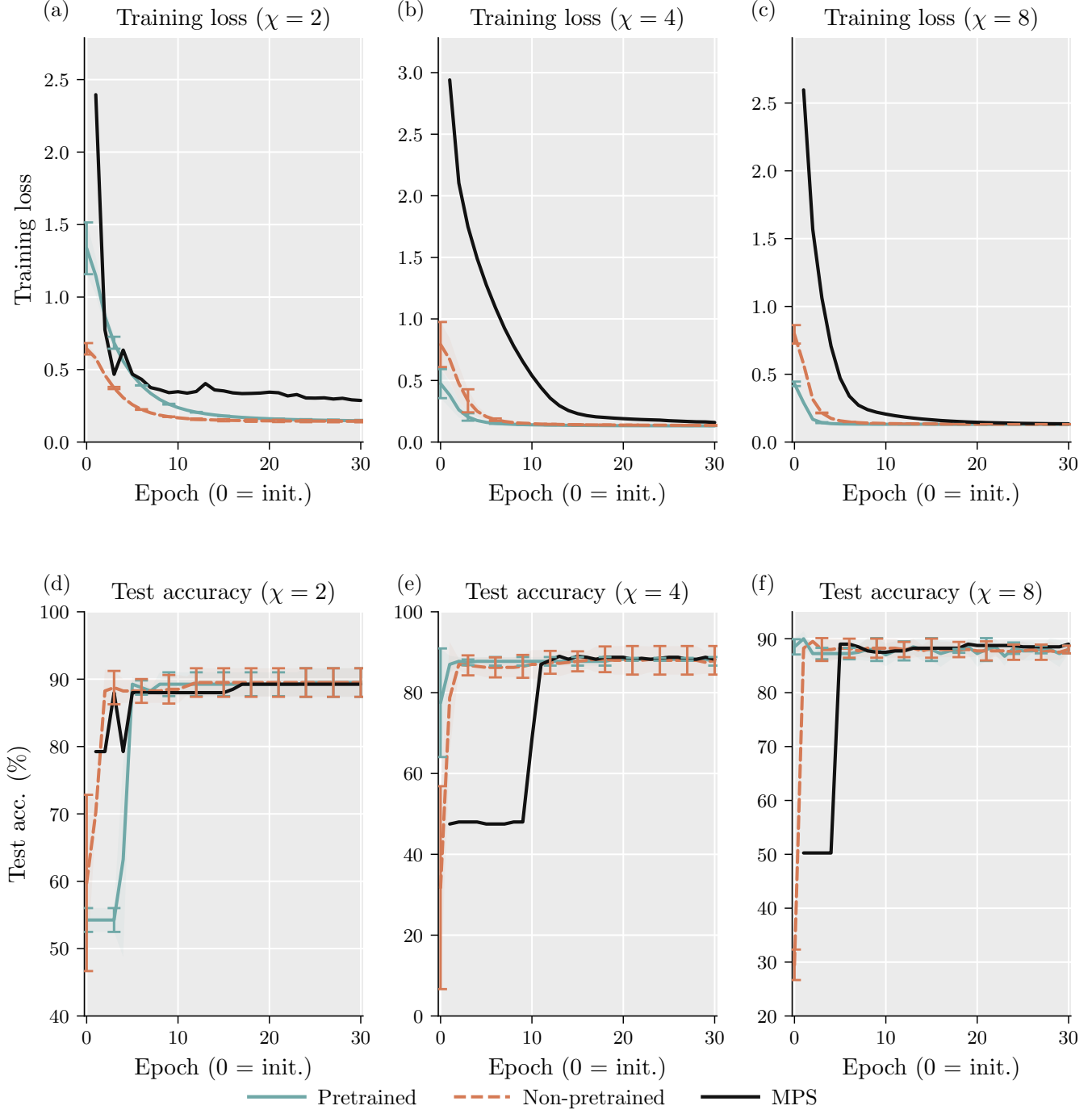


Figure 3.3: Training loss and test accuracy for the dense $SO(n)$ parameterisation with bond dimensions $\chi = 2, 4, 8$. The classically trained MPS is compared with the corresponding pretrained VQC and a non-pretrained VQC with the same circuit architecture. Epoch zero denotes the VQC performance immediately after initialisation, before any VQC optimisation is performed. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.

out classically by the MPS.

A similar behaviour is observed for $\chi = 8$. The pretrained VQC again starts with a substantially lower loss and higher test accuracy than the non-pretrained model. The non-pretrained VQC improves rapidly during the first few epochs and eventually reaches a similar accuracy. For the Bars-and-Stripes task considered here, increasing the bond dimension does not lead to a higher final VQC accuracy, but the larger- χ models provide a clearer hot-start advantage from MPS pretraining. This behaviour is consistent with the increased capacity and number of trainable degrees of freedom available during the classical pretraining stage.

An important observation is that the MPS and the pretrained VQC do not necessarily have identical loss or accuracy immediately after the parameter transfer. This does not mean that the transfer is approximate. The parameters themselves are transferred directly. The difference comes from the way the two models use the corresponding matrices. The MPS uses selected parts of the parent orthogonal matrices during its contractions, whereas the VQC applies the complete orthogonal matrices and therefore evolves additional amplitudes that are not retained by the MPS. As a result, the same parameters can produce different complete output probabilities in the two models. Overall, these results are consistent with the main goal of the project: giving the VQC a pre-training advantage to reduce the number of optimisation steps done by it.

The repeated runs also indicate that this hot-start behaviour is not restricted to a single random initialisation. For $\chi = 4$ and $\chi = 8$, the pretrained VQC begins in a lower-loss region on average across the repeated experiments, supporting the robustness of the observed pretraining advantage.

3.2.2 Adding trainable layers to the pretrained VQC

The previous experiment investigated whether the parameters learned by the MPS provide a useful initialisation for the VQC. We now consider a second question. Although the MPS provides the initial parameters of the quantum circuit, the resulting VQC does not have to remain restricted to the model that can be represented by the MPS. Once the parameters have been transferred, additional trainable quantum gates can be added to increase the expressive power of the VQC [36].

We perform this experiment using the same binary Bars-and-Stripes classification task as in the previous section. The same training and test datasets, data encoding, and optimisation procedure are used. We fix the bond dimension to $\chi = 4$, for which the previous experiment showed a clear advantage from MPS pretraining.

After training the MPS classically, its parameters are transferred to the corresponding gates of the VQC as before. We then extend the pretrained circuit by adding additional trainable layers after the MPS-derived part of the circuit. We denote the number of additional layers by L , and consider

$$L = 0, 1, 2, 3.$$

The case $L = 0$ corresponds to the direct MPS-to-VQC mapping, with no additional

gates. The cases $L = 1$, $L = 2$, and $L = 3$ contain an increasing number of trainable two-qubit $SO(4)$ gates after the pretrained circuit. Each $SO(4)$ gate is parameterised using the same dense Givens-rotation parameterisation introduced previously.

The parameters of the additional gates are initialised close to zero, so that the corresponding $SO(4)$ matrices are close to the identity. The reason for this choice is that we want to increase the expressive power of the VQC without completely destroying the information already learned by the MPS [36]. At initialisation, the extended VQC therefore remains related to the pretrained circuit, while the additional parameters are free to move away from the identity during VQC training.

This experiment allows us to investigate the role of the MPS pretraining more clearly. The MPS provides the initial structured part of the quantum circuit, while the additional layers give the VQC degrees of freedom that were not available during classical MPS training. We can therefore test whether the VQC only needs to refine the pretrained solution, and whether increasing its expressivity after the parameter transfer leads to a lower training loss.

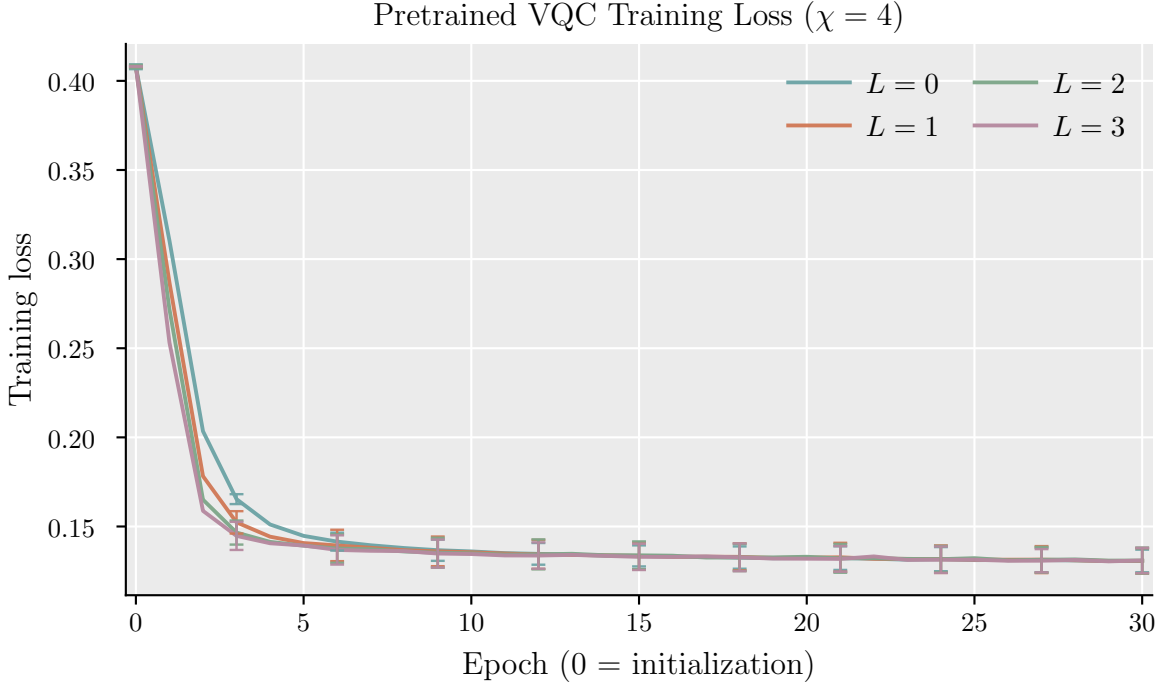


Figure 3.4: Training loss of the pretrained VQC with bond dimension $\chi = 4$ for different numbers of additional trainable layers $L = 0, 1, 2, 3$. The case $L = 0$ corresponds to the direct MPS-to-VQC mapping, while $L > 0$ introduces additional trainable two-qubit $SO(4)$ gates initialised close to the identity. All models therefore begin from approximately the same loss, while the additional layers allow the VQC to reduce the loss more rapidly during the first few optimisation epochs. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.

Results and discussion

Figure 3.4 shows the training loss of the pretrained VQC for different numbers of additional trainable layers. At epoch zero, all four models begin with approximately the same training loss. This is expected because the parameters of the additional $SO(4)$ gates are initialised close to zero, making the corresponding gates close to the identity. Therefore, before VQC optimisation begins, adding the extra layers only slightly changes the circuit obtained from the MPS parameter transfer.

Once VQC training begins, however, a clear difference can be observed. The circuit with $L = 0$ reduces its loss more slowly during the first few epochs, while the circuits with $L = 1$, $L = 2$, and $L = 3$ reach the low-loss region more rapidly. For example, around epoch 4, the $L = 0$ circuit still has a noticeably larger loss, while the circuits containing additional layers have already reached losses close to the final region.

This behaviour suggests that the additional layers provide useful trainable degrees of freedom after the MPS parameters have been transferred. The MPS pretraining determines the initial point of the optimisation, while the additional gates allow the VQC to move beyond the parameter space represented by the original MPS-derived circuit. The VQC can therefore refine the pretrained solution using parameters that were not available during classical MPS training.

The difference between $L = 0$ and the circuits containing additional layers is visible, while the differences between $L = 1$, $L = 2$, and $L = 3$ are considerably smaller. Increasing the number of additional layers therefore does not produce a proportional improvement in the training loss.

After several epochs, all four models approach a similar low-loss region. This indicates that the main benefit of the additional layers in this experiment is faster refinement of the pretrained solution rather than a large improvement in the final loss. In particular, adding one or a small number of layers already provides most of the observed benefit, while further increasing the circuit depth gives only a small additional improvement.

Chapter 4

Efficient parameterisation

The dense $SO(n)$ parameterisation introduced in the previous chapter provides a direct parameter correspondence between the MPS and the VQC. However, it also introduces an important practical limitation. The corresponding quantum gates are general dense orthogonal transformations acting on several qubits and cannot in general be implemented directly on quantum hardware. They must first be decomposed into sequences of elementary one- and two-qubit gates.

The decomposition of a general multi-qubit transformation can lead to a large number of elementary gates and consequently deep quantum circuits. In particular, quantum-circuit synthesis methods for generic q -qubit operators have gate counts which grow exponentially with the number of qubits [41]. This makes the use of general dense transformations less attractive as the size of the local gates increases. The synthesis results in [41] show that generic multi-qubit operators require a rapidly increasing number of two-qubit gates, with asymptotic CNOT counts scaling with 4^q .

In our dense parameterisation, a gate acting on q qubits is represented by a matrix of dimension

$$n = 2^q. \tag{4.1}$$

Although the dense $SO(n)$ matrices provide a convenient way of establishing the direct MPS-to-VQC parameter transfer, implementing these matrices on quantum hardware could result in deep circuit.

We therefore take a different approach. Instead of first constructing a dense $SO(n)$ matrix, we define the elementary quantum-gate structure from the beginning. The matrix generated by this circuit is then used as the parent matrix from which the MPS tensor is constructed.

Each local block consists of L_c layers. Within each layer, we first apply a trainable R_y rotation to every qubit,

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}. \tag{4.2}$$

These rotations are followed by trainable nearest-neighbour controlled- R_y gates, defined as

$$CR_y(\phi) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes R_y(\phi). \quad (4.3)$$

The direction of the controlled-rotation ladder is alternated between successive layers: one layer acts from left to right and the next from right to left.

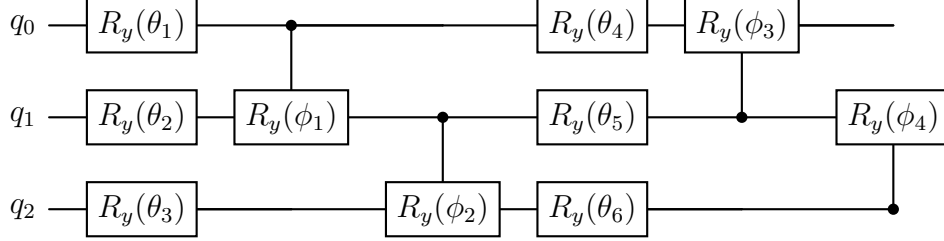


Figure 4.1: Example of two layers of the structured circuit used to generate an efficient parent matrix. Each layer consists of trainable single-qubit R_y rotations followed by nearest-neighbour controlled- R_y rotations. The direction of the controlled-rotation ladder alternates between successive layers.

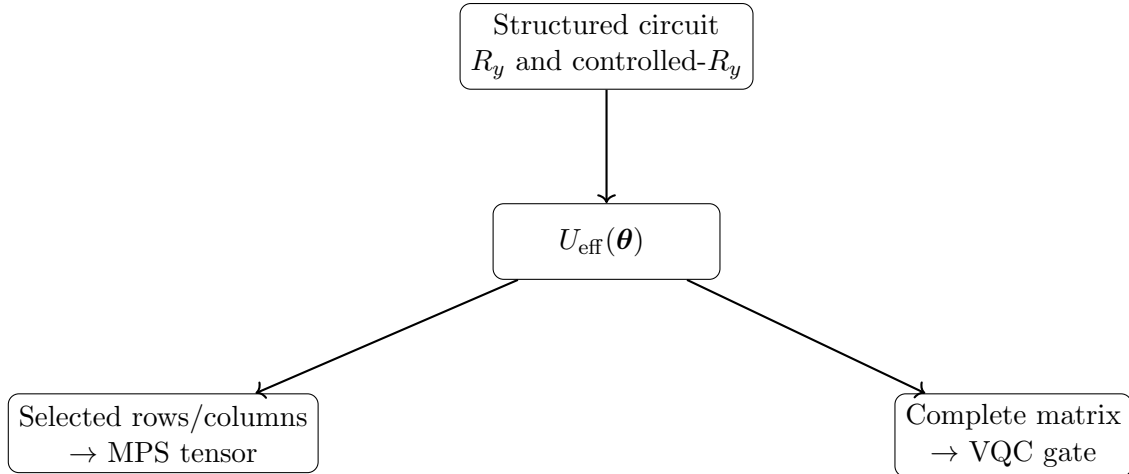


Figure 4.2: Direct parameter-transfer construction for the efficient parameterisation. The structured circuit generates the parent matrix $U_{\text{eff}}(\boldsymbol{\theta})$. Selected rows or columns are used to construct the corresponding MPS tensor, while the complete matrix is used as the VQC gate. Both therefore depend on the same parameters $\boldsymbol{\theta}$.

For a block acting on q qubits, each layer contains q single-qubit R_y rotations and $q - 1$ nearest-neighbour controlled- R_y gates. The number of gates in one layer is therefore

$$N_{\text{layer}} = q + (q - 1) = 2q - 1. \quad (4.4)$$

For L_c layers, the structured block contains

$$N_{\text{gates}} = L_c(2q - 1) = \mathcal{O}(L_c q) \quad (4.5)$$

gates. Therefore, for a fixed number of layers, the size of the circuit grows linearly with the number of qubits.

The main advantage of the structured parameterisation is that the quantum gate structure is fixed from the beginning. The same R_y and controlled- R_y gates used to generate the MPS parent matrices can be used directly in the VQC.

The MPS tensors are then constructed in the same way as for the dense parameterisation. The first tensor uses selected columns of the parent matrix, the intermediate tensors use selected rows, and the final tensor uses the complete matrix. The difference is that the parent matrix is now generated by the structured circuit rather than being a general dense $SO(n)$ matrix.

4.1 Experiments

We now test whether the efficient parameterisation can preserve the pretraining advantage observed with the dense $SO(n)$ parameterisation. To make the comparison as direct as possible, we use the same Bars-and-Stripes classification task and the same data encoding and training procedure as in Section 3.2.

Here, the bond dimension is fixed to $\chi = 4$. Instead of changing the bond dimension, we vary the number of layers L_c used inside each efficient circuit block. We consider

$$L_c = 2, 4, 6.$$

Increasing L_c increases the number of trainable parameters and therefore the expressivity of the structured parent matrices.

For each value of L_c , the MPS is first trained classically. Its optimised parameters are then transferred directly to the corresponding VQC, giving the pretrained VQC. As in the previous chapter, we compare this against a VQC with the same architecture but without MPS pretraining.

This experiment therefore tests whether the hot-start behaviour found with the dense parameterisation can still be obtained when the parent matrices are restricted to a much more structured and efficient circuit family.

Results and discussion

Figure 4.3 shows that the behaviour of the efficient parameterisation depends strongly on the number of circuit layers L_c .

For $L_c = 2$, there is no clear advantage from MPS pretraining at initialization. The pretrained VQC does not begin with a lower training loss than the non-pretrained VQC, and the two models have similar initial test accuracy. After several optimisation epochs, both VQCs reach a similar region. This suggests that, for the task considered here, two

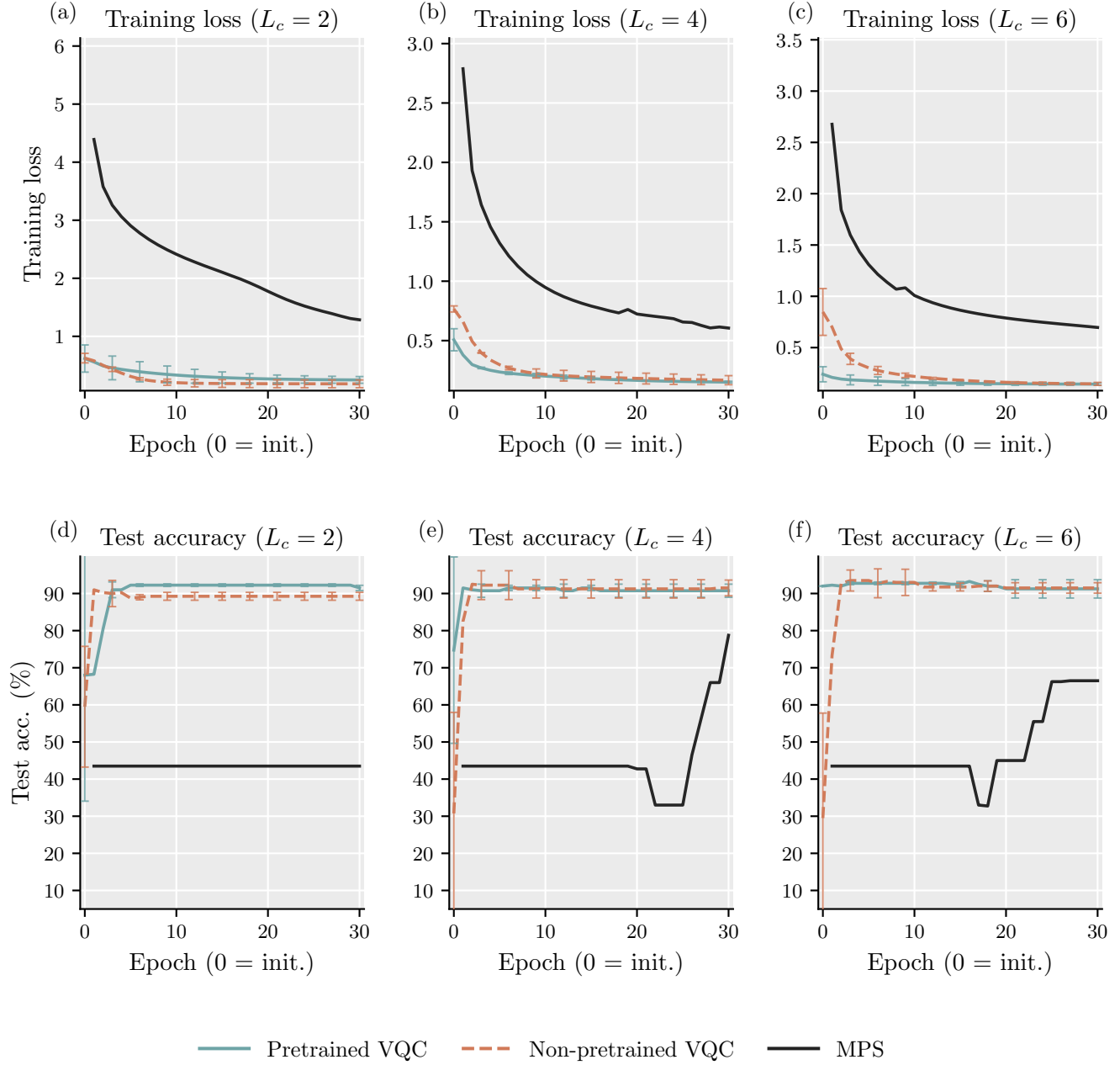


Figure 4.3: Training loss and test accuracy for the efficient parameterisation with $L_c = 2, 4, 6$ and fixed bond dimension $\chi = 4$. Results are shown for the classically trained MPS, the pretrained VQC, and the non-pretrained VQC with the same circuit architecture. Epoch zero denotes the VQC performance immediately after initialisation and before any VQC optimisation. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.

circuit layers provide insufficient flexibility for the MPS to learn parameters that give a consistently useful VQC initialisation.

The behaviour changes when the number of layers is increased. For $L_c = 4$, the pretrained VQC starts with a higher test accuracy and a lower training loss than the non-pretrained VQC. The non-pretrained circuit is still able to learn the task quickly, but requires more optimisation steps to reach the same region. The effect becomes even clearer for $L_c = 6$, where the pretrained VQC already starts close to the low-loss and high-accuracy region reached after training.

Increasing L_c gives each local circuit block more trainable parameters and therefore increases the expressivity of the MPS parameterisation. The results suggest that this additional expressivity is important for the classical pretraining stage. If the circuit blocks are too shallow, as for $L_c = 2$, the MPS is more restricted and the transferred parameters are less useful. Once sufficient circuit depth is available, the MPS is able to provide a much stronger starting point for the VQC.

This behaviour is similar to what was observed for the dense parameterisation in Chapter 3. There, the pretraining advantage was limited for the smallest bond dimension $\chi = 2$, while a clearer hot start was obtained for $\chi = 4$ and $\chi = 8$. In the present experiment the bond dimension is fixed, but the same general trade-off appears through L_c : increasing the capacity of the MPS improves the quality of the transferred initialisation.

There is, however, an important difference from the dense case. The MPS training losses obtained with the efficient parameterisation remain higher, particularly for the smaller values of L_c . This is expected because the parent matrices are no longer allowed to explore the complete dense $SO(n)$ space. They are restricted to matrices that can be generated by the chosen shallow circuit structure. The efficient parameterisation therefore trades some expressivity for a much simpler quantum implementation.

Despite this restriction, the results for $L_c = 4$ and $L_c = 6$ show that the main pretraining effect is preserved. The pretrained VQC begins close to a good solution and the non-pretrained VQC reaches a similar region only after quantum optimisation. This is an important result because it shows that the hot start observed in the dense experiments does not require a fully dense $SO(n)$ parameterisation.

The efficient circuit can therefore preserve the main advantage of the MPS pretraining while using a structured gate family whose circuit size grows only linearly with the number of qubits for fixed L_c . The results suggest that L_c must be large enough to provide sufficient expressivity, but it does not need to reproduce a completely general dense orthogonal transformation.

The repeated runs show that this behaviour is also robust to random initialisation. In particular, for $L_c = 4$ and $L_c = 6$, the pretrained VQC starts in a lower-loss region on average across the repeated experiments, whereas the advantage is less clear for $L_c = 2$.

4.1.1 Additional trainable VQC layers

We finally repeat the additional-layer experiment of Section 3.2.2 using the efficient parameterisation. We fix $L_c = 6$ and add $L = 0, 1, 2, 3$ trainable layers after the MPS-

derived part of the VQC. As before, the additional gates are initialised close to the identity so that they do not significantly change the pretrained solution at initialization. The aim is to test whether the VQC can still benefit from additional trainable freedom when the MPS itself is parameterised by the more restricted efficient circuit structure.

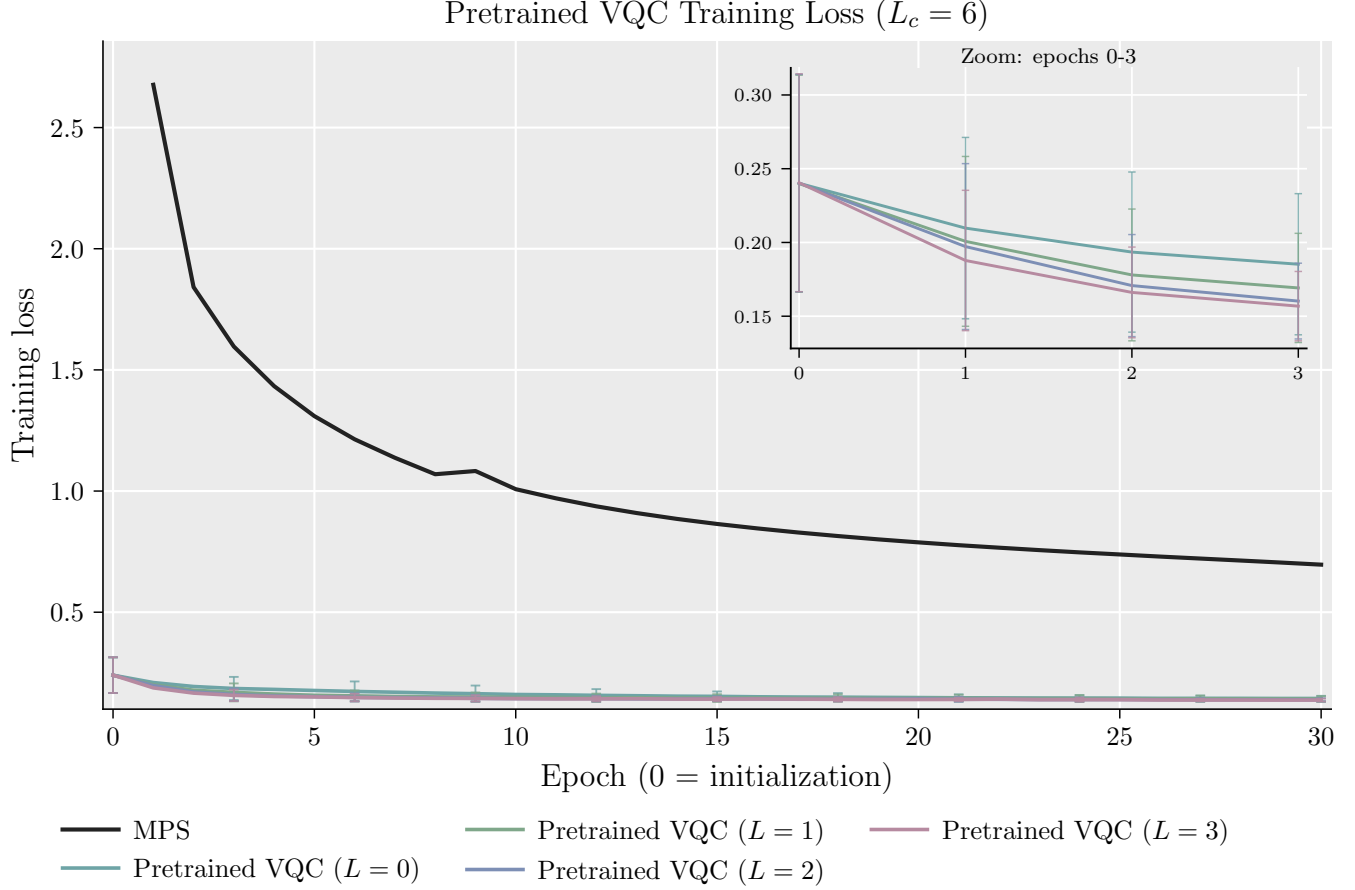


Figure 4.4: Training loss for the efficient parameterisation with $L_c = 6$ and additional trainable VQC layers $L = 0, 1, 2, 3$. All pretrained VQCs begin from approximately the same loss because the additional layers are initialised close to the identity. The additional layers reduce the loss more rapidly during the first few optimisation epochs, while all models eventually converge to a similar low-loss region. The inset highlights the early stages of training. Curves show the mean over 5 independent runs, with error bars indicating one standard deviation at selected epochs.

Results and discussion

Figure 4.4 shows a behaviour similar to that observed for the dense parameterisation. At epoch zero, the pretrained VQCs have almost identical training losses. This is expected because the additional layers are initialised close to the identity.

Once training begins, the circuits with additional layers reduce the loss more quickly than the $L = 0$ circuit. The difference is most visible during the first few epochs, as shown in the inset. Increasing L from 0 to 1, 2, and 3 gives progressively more freedom to refine the pretrained solution, although the differences between the three extended circuits are relatively small.

After several epochs, all four VQCs approach almost the same low-loss region. As in the dense experiment, the main benefit of the additional layers is therefore faster refinement of the pretrained solution rather than a large improvement in the final loss.

Compared with the dense-parameterisation experiment in Figure 3.4, the $L_c = 6$ efficient VQC in Figure 4.4 begins from a lower training loss. The differences between the curves for different values of L are also smaller. This suggests that, for this experiment, the parameters learned by the $L_c = 6$ efficient MPS already provide a strong initialisation, leaving less room for the additional trainable layers to improve the circuit during the early stages of VQC optimisation.

The MPS loss itself remains noticeably higher than the VQC loss. As discussed in the previous chapter, this does not imply that the parameter transfer is approximate. The same parameters are transferred directly, but the MPS and VQC use the corresponding transformations differently and therefore do not necessarily produce the same output probabilities.

Overall, Figures 4.3 and 4.4 show that the main behaviour observed with the dense $SO(n)$ parameterisation is preserved after replacing the dense blocks by the efficient circuit parameterisation. Classical MPS pretraining can still provide a useful VQC initialisation, and additional quantum layers can still be used to further reduce the number of optimisation steps.

Chapter 5

Conclusion

The main goal of this project was to investigate whether an MPS can be used to classically pretrain a variational quantum classifier. The idea was to perform part of the optimisation using a classical tensor-network model and then transfer the learned parameters to the quantum circuit, giving the VQC a better starting point for its own optimisation.

To achieve this, we parameterised the MPS so that its parameters correspond directly to parameters of the VQC. This allows the MPS to be trained classically and its optimised parameters transferred directly to the quantum circuit without requiring an intermediate decomposition or parameter-matching procedure.

We first tested this idea using dense $SO(n)$ matrices. The numerical experiments showed that the usefulness of the pretraining depends on the capacity of the MPS. For the smallest bond dimension considered, $\chi = 2$, no clear hot-start advantage was observed. For $\chi = 4$ and $\chi = 8$, however, the pretrained VQC started much closer to a low-loss and high-accuracy region. The randomly initialised VQC was still able to reach a similar solution, but required more VQC optimisation to do so.

This is an important distinction. The aim of the pretraining is not necessarily to obtain a higher final classification accuracy. Instead, the aim is to perform useful optimisation classically before the quantum model is trained. The results show that, when the MPS is expressive enough, its trained parameters can provide this hot start. Repeating the experiments across different random initialisations showed that this hot-start behaviour is also observed on average across independent runs, providing evidence that the result is not specific to a single initialisation.

We also investigated what happens when additional trainable gates are added to the VQC after the parameter transfer. These gates were initialised close to the identity so that the pretrained solution was approximately preserved. During training, the additional gates allowed the VQC to refine this solution more quickly. However, increasing the number of additional layers only gave a small further improvement for the problem considered here.

Although the dense parameterisation was useful to demonstrate the main idea, it also has an important practical limitation. General dense multi-qubit transformations have to be decomposed into elementary quantum gates before they can be implemented on quantum hardware. This can require a large number of gates and therefore produce deep

quantum circuits.

For this reason, we introduced a second parameterisation in which the MPS parent matrices are generated directly from layers of R_y and controlled- R_y gates. For a fixed number of circuit layers, the number of gates in each structured block grows linearly with the number of qubits.

The results showed that this more structured parameterisation can still preserve the main pretraining behaviour. When the circuit is too shallow, the MPS does not have enough freedom to provide a useful initialisation. As the number of circuit layers is increased, the quality of the pretraining improves. This suggests that the circuit needs to be expressive enough, but it does not need to reproduce a completely general dense orthogonal transformation.

There are also several limitations to the experiments carried out in this project. The main experiments were performed on a small 2×2 Bars-and-Stripes classification problem. This was useful as a proof-of-concept, but it is still a relatively simple task. It would therefore be important to test the method on more datasets and on larger classification problems to see whether the same behaviour is still observed.

The experiments were also performed using classical simulation of the quantum circuits. During training, the circuits were evaluated exactly without finite-shot sampling. Therefore, the results do not directly show the computational advantage that could be obtained on real quantum hardware. Future work should also study the method with finite-shot training, noise, and eventually on real quantum devices.

There are also many other possible extensions of the method presented here. In this project, we only considered two particular ways of parameterising the MPS and VQC. Other MPS architectures could be used, and more generally other tensor network structures could also be investigated. Different tensor-network architectures may allow different quantum-circuit structures and different ways of carrying out the direct parameter transfer. The method is also not restricted to the exact learning setup used here. Different feature maps, loss functions, measurements, and optimisation procedures could be used.

Overall, this project shows that it is possible to classically pretrain a tensor network and directly transfer its parameters to a variational quantum circuit without requiring an intermediate decomposition step.

Appendix A

Three-Site MPS Example

Consider a three-site MPS with physical dimension $d = 2$, bond dimension $\chi = 4$, and output dimension 2. The three tensors have the dimensions

$$A^{[1]} \in \mathbb{R}^{4 \times 2}, \quad A^{[2]} \in \mathbb{R}^{4 \times 4 \times 2}, \quad A^{[3]} \in \mathbb{R}^{2 \times 4 \times 4 \times 2}. \quad (\text{A.1})$$

The first tensor has one physical index and one bond index. The middle tensor has one incoming bond index, one outgoing bond index, and one physical index. In the implementation used in this work, the last tensor has one incoming bond index, one physical index, one class index, and one auxiliary output index. The class index has dimension two and the auxiliary index has dimension four.

For each site i , the scalar input x_i is mapped to the two-dimensional feature

$$\phi(x_i) = \begin{pmatrix} \phi_0(x_i) \\ \phi_1(x_i) \end{pmatrix} \in \mathbb{R}^2. \quad (\text{A.2})$$

The physical index of each MPS tensor is contracted with one of these local feature vectors. The contraction begins at the first site, moves through the middle site, and finishes at the last site.

Contraction of the first site. The first tensor has dimensions 4×2 , and can therefore be written as

$$A^{[1]} = \begin{pmatrix} a_{10}^{[1]} & a_{11}^{[1]} \\ a_{20}^{[1]} & a_{21}^{[1]} \\ a_{30}^{[1]} & a_{31}^{[1]} \\ a_{40}^{[1]} & a_{41}^{[1]} \end{pmatrix}. \quad (\text{A.3})$$

The rows correspond to the bond index, which has dimension $\chi = 4$, while the columns correspond to the physical index, which has dimension $d = 2$.

We contract the physical index of $A^{[1]}$ with the feature vector of the first site:

$$\mathbf{h}^{[1]} = A^{[1]} \phi(x_1). \quad (\text{A.4})$$

Writing this contraction explicitly gives

$$\begin{pmatrix} h_1^{[1]} \\ h_2^{[1]} \\ h_3^{[1]} \\ h_4^{[1]} \end{pmatrix} = \begin{pmatrix} a_{10}^{[1]} & a_{11}^{[1]} \\ a_{20}^{[1]} & a_{21}^{[1]} \\ a_{30}^{[1]} & a_{31}^{[1]} \\ a_{40}^{[1]} & a_{41}^{[1]} \end{pmatrix} \begin{pmatrix} \phi_0(x_1) \\ \phi_1(x_1) \end{pmatrix}. \quad (\text{A.5})$$

Therefore,

$$\mathbf{h}^{[1]} = \begin{pmatrix} a_{10}^{[1]} \phi_0(x_1) + a_{11}^{[1]} \phi_1(x_1) \\ a_{20}^{[1]} \phi_0(x_1) + a_{21}^{[1]} \phi_1(x_1) \\ a_{30}^{[1]} \phi_0(x_1) + a_{31}^{[1]} \phi_1(x_1) \\ a_{40}^{[1]} \phi_0(x_1) + a_{41}^{[1]} \phi_1(x_1) \end{pmatrix}. \quad (\text{A.6})$$

The dimensions involved in this operation are

$$(4 \times 2)(2 \times 1) = (4 \times 1). \quad (\text{A.7})$$

The result is therefore a four-dimensional vector. This vector contains the information obtained from the first input after it has been mapped into the bond space. We call it the intermediate bond vector $\mathbf{h}^{[1]}$.

Structure of the middle tensor. The middle tensor has dimensions

$$A^{[2]} \in \mathbb{R}^{4 \times 4 \times 2}. \quad (\text{A.8})$$

We denote its elements by

$$A_{\beta\alpha s}^{[2]}, \quad (\text{A.9})$$

where

- $\alpha \in \{1, 2, 3, 4\}$ is the incoming bond index,
- $\beta \in \{1, 2, 3, 4\}$ is the outgoing bond index,
- $s \in \{0, 1\}$ is the physical index.

The incoming bond index is contracted with the vector $\mathbf{h}^{[1]}$, while the physical index is contracted with the feature vector $\phi(x_2)$. The outgoing bond index is not contracted. It becomes the index of the new bond vector $\mathbf{h}^{[2]}$.

Contraction for one outgoing bond component. To understand the middle contraction, we first fix the outgoing bond index β . For a fixed value of β , the tensor becomes a 4×2 matrix:

$$A_{\beta}^{[2]} = \begin{pmatrix} A_{\beta 10}^{[2]} & A_{\beta 11}^{[2]} \\ A_{\beta 20}^{[2]} & A_{\beta 21}^{[2]} \\ A_{\beta 30}^{[2]} & A_{\beta 31}^{[2]} \\ A_{\beta 40}^{[2]} & A_{\beta 41}^{[2]} \end{pmatrix}. \quad (\text{A.10})$$

The rows correspond to the incoming bond index α , and the columns correspond to the physical index s .

One component $h_{\beta}^{[2]}$ of the new bond vector is obtained by contracting the incoming bond vector from the left and the local feature vector from the right:

$$h_{\beta}^{[2]} = \begin{pmatrix} h_1^{[1]} & h_2^{[1]} & h_3^{[1]} & h_4^{[1]} \end{pmatrix} A_{\beta}^{[2]} \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}. \quad (\text{A.11})$$

The dimensions are

$$(1 \times 4)(4 \times 2)(2 \times 1) = (1 \times 1). \quad (\text{A.12})$$

The result is therefore one scalar. This scalar is one component of the new four-dimensional bond vector.

For example, let us calculate the first component, corresponding to $\beta = 1$. The relevant matrix is

$$A_1^{[2]} = \begin{pmatrix} A_{110}^{[2]} & A_{111}^{[2]} \\ A_{120}^{[2]} & A_{121}^{[2]} \\ A_{130}^{[2]} & A_{131}^{[2]} \\ A_{140}^{[2]} & A_{141}^{[2]} \end{pmatrix}. \quad (\text{A.13})$$

We then have

$$h_1^{[2]} = \begin{pmatrix} h_1^{[1]} & h_2^{[1]} & h_3^{[1]} & h_4^{[1]} \end{pmatrix} \begin{pmatrix} A_{110}^{[2]} & A_{111}^{[2]} \\ A_{120}^{[2]} & A_{121}^{[2]} \\ A_{130}^{[2]} & A_{131}^{[2]} \\ A_{140}^{[2]} & A_{141}^{[2]} \end{pmatrix} \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}. \quad (\text{A.14})$$

The matrix is first contracted with the local feature vector:

$$A_1^{[2]} \phi(x_2) = \begin{pmatrix} A_{110}^{[2]} \phi_0(x_2) + A_{111}^{[2]} \phi_1(x_2) \\ A_{120}^{[2]} \phi_0(x_2) + A_{121}^{[2]} \phi_1(x_2) \\ A_{130}^{[2]} \phi_0(x_2) + A_{131}^{[2]} \phi_1(x_2) \\ A_{140}^{[2]} \phi_0(x_2) + A_{141}^{[2]} \phi_1(x_2) \end{pmatrix}. \quad (\text{A.15})$$

The resulting vector is then contracted with $\mathbf{h}^{[1]}$, giving

$$\begin{aligned} h_1^{[2]} = & h_1^{[1]} \left(A_{110}^{[2]} \phi_0(x_2) + A_{111}^{[2]} \phi_1(x_2) \right) \\ & + h_2^{[1]} \left(A_{120}^{[2]} \phi_0(x_2) + A_{121}^{[2]} \phi_1(x_2) \right) \\ & + h_3^{[1]} \left(A_{130}^{[2]} \phi_0(x_2) + A_{131}^{[2]} \phi_1(x_2) \right) \\ & + h_4^{[1]} \left(A_{140}^{[2]} \phi_0(x_2) + A_{141}^{[2]} \phi_1(x_2) \right). \end{aligned} \quad (\text{A.16})$$

The same procedure is repeated for each value of the outgoing bond index $\beta \in \{1, 2, 3, 4\}$. Thus,

$$h_2^{[2]} = \begin{pmatrix} h_1^{[1]} & h_2^{[1]} & h_3^{[1]} & h_4^{[1]} \end{pmatrix} A_2^{[2]} \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}, \quad (\text{A.17})$$

$$h_3^{[2]} = \begin{pmatrix} h_1^{[1]} & h_2^{[1]} & h_3^{[1]} & h_4^{[1]} \end{pmatrix} A_3^{[2]} \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}, \quad (\text{A.18})$$

and

$$h_4^{[2]} = \begin{pmatrix} h_1^{[1]} & h_2^{[1]} & h_3^{[1]} & h_4^{[1]} \end{pmatrix} A_4^{[2]} \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}. \quad (\text{A.19})$$

These four scalars are collected into the new bond vector

$$\mathbf{h}^{[2]} = \begin{pmatrix} h_1^{[2]} \\ h_2^{[2]} \\ h_3^{[2]} \\ h_4^{[2]} \end{pmatrix} \in \mathbb{R}^4. \quad (\text{A.20})$$

Therefore, each fixed value of the outgoing bond index selects one 4×2 matrix from the middle tensor. Contracting this matrix with the incoming bond vector and the local feature vector produces one scalar. Repeating the operation four times produces the complete outgoing bond vector.

The same middle contraction can be written more compactly using index notation:

$$h_\beta^{[2]} = \sum_{\alpha=1}^4 \sum_{s=0}^1 A_{\beta\alpha s}^{[2]} h_\alpha^{[1]} \phi_s(x_2). \quad (\text{A.21})$$

The sums over α and s represent the contraction of the incoming bond index and the physical index, respectively. The index β remains open and labels the components of the outgoing bond vector.

Contraction of the final site. The final site is treated slightly differently from the middle site. The incoming bond vector still has dimension $\chi = 4$, and the local feature vector still has dimension $d = 2$. Their tensor product therefore has dimension

$$\chi d = 4 \times 2 = 8. \quad (\text{A.22})$$

In the implementation used in this work, the complete final $SO(8)$ matrix is retained rather than selecting only two rows. The last tensor therefore has dimensions

$$A^{[3]} \in \mathbb{R}^{2 \times 4 \times 4 \times 2}. \quad (\text{A.23})$$

We denote its elements by

$$A_{oe\alpha s}^{[3]}, \quad (\text{A.24})$$

where

- $o \in \{0, 1\}$ is the class index,
- $e \in \{1, 2, 3, 4\}$ is an auxiliary output index,
- $\alpha \in \{1, 2, 3, 4\}$ is the incoming bond index,
- $s \in \{0, 1\}$ is the physical index.

The two input indices (α, s) can be grouped into one eight-dimensional column index, while the two output indices (o, e) can be grouped into one eight-dimensional row index. Thus the last tensor can be viewed directly as the matrix

$$U_{(o,e),(\alpha,s)}^{[3]} \in \mathbb{R}^{8 \times 8}. \quad (\text{A.25})$$

For later convenience, define the row vector

$$\mathbf{a}_{oe}^{[3]} = \left(A_{oe10}^{[3]} \ A_{oe11}^{[3]} \ A_{oe20}^{[3]} \ A_{oe21}^{[3]} \ A_{oe30}^{[3]} \ A_{oe31}^{[3]} \ A_{oe40}^{[3]} \ A_{oe41}^{[3]} \right). \quad (\text{A.26})$$

With the row ordering

$$(o, e) = (0, 1), (0, 2), (0, 3), (0, 4), (1, 1), (1, 2), (1, 3), (1, 4), \quad (\text{A.27})$$

the complete final matrix is

$$U^{[3]} = \begin{pmatrix} \mathbf{a}_{01}^{[3]} \\ \mathbf{a}_{02}^{[3]} \\ \mathbf{a}_{03}^{[3]} \\ \mathbf{a}_{04}^{[3]} \\ \mathbf{a}_{11}^{[3]} \\ \mathbf{a}_{12}^{[3]} \\ \mathbf{a}_{13}^{[3]} \\ \mathbf{a}_{14}^{[3]} \end{pmatrix} \in SO(8). \quad (\text{A.28})$$

The input to this matrix is the tensor product of the bond vector produced by the middle site and the feature vector of the final site:

$$\mathbf{z}^{[3]} = \mathbf{h}^{[2]} \otimes \phi(x_3) = \begin{pmatrix} h_1^{[2]} \phi_0(x_3) \\ h_1^{[2]} \phi_1(x_3) \\ h_2^{[2]} \phi_0(x_3) \\ h_2^{[2]} \phi_1(x_3) \\ h_3^{[2]} \phi_0(x_3) \\ h_3^{[2]} \phi_1(x_3) \\ h_4^{[2]} \phi_0(x_3) \\ h_4^{[2]} \phi_1(x_3) \end{pmatrix} \in \mathbb{R}^8. \quad (\text{A.29})$$

The final contraction is therefore simply

$$\mathbf{y} = U^{[3]} \mathbf{z}^{[3]} \in \mathbb{R}^8. \quad (\text{A.30})$$

Writing the output explicitly,

$$\mathbf{y} = \begin{pmatrix} y_{01} \\ y_{02} \\ y_{03} \\ y_{04} \\ y_{11} \\ y_{12} \\ y_{13} \\ y_{14} \end{pmatrix} = \begin{pmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \end{pmatrix}, \quad \mathbf{y}_0, \mathbf{y}_1 \in \mathbb{R}^4. \quad (\text{A.31})$$

The first four amplitudes correspond to class 0, and the last four amplitudes correspond to class 1. The class index is therefore not represented by a single amplitude. Instead, each class is associated with a four-dimensional block of amplitudes.

The MPS forms one weight for each class by first taking the squared magnitude of the amplitudes and then summing over the auxiliary index:

$$w_0 = \sum_{e=1}^4 |y_{0e}|^2 = \|\mathbf{y}_0\|^2, \quad (\text{A.32})$$

and

$$w_1 = \sum_{e=1}^4 |y_{1e}|^2 = \|\mathbf{y}_1\|^2. \quad (\text{A.33})$$

The final MPS probabilities are

$$p_0 = \frac{w_0}{w_0 + w_1}, \quad p_1 = \frac{w_1}{w_0 + w_1}. \quad (\text{A.34})$$

Equivalently, in index notation the environment-resolved amplitudes are

$$y_{oe} = \sum_{\alpha=1}^4 \sum_{s=0}^1 A_{oe\alpha s}^{[3]} h_{\alpha}^{[2]} \phi_s(x_3), \quad (\text{A.35})$$

and the class probabilities are obtained from

$$p_o = \frac{\sum_{e=1}^4 |y_{oe}|^2}{\sum_{o'=0}^1 \sum_{e=1}^4 |y_{o'e}|^2}, \quad o \in \{0, 1\}. \quad (\text{A.36})$$

This differs from the simpler two-logit construction in which the final tensor has shape $2 \times 4 \times 2$ and produces only two numbers y_0 and y_1 . The implementation used in this work deliberately keeps the complete 8×8 final matrix. It therefore produces eight amplitudes and obtains the two class probabilities by grouping and summing their squared magnitudes.

Summary of the contraction. The complete three-site contraction is therefore performed in three stages:

1. The first tensor contracts with $\phi(x_1)$ to produce the initial four-dimensional bond vector $\mathbf{h}^{[1]}$.
2. The middle tensor contracts with both $\mathbf{h}^{[1]}$ and $\phi(x_2)$ to produce a new four-dimensional bond vector $\mathbf{h}^{[2]}$.
3. The final 8×8 matrix acts on $\mathbf{h}^{[2]} \otimes \phi(x_3)$ and produces eight amplitudes. These amplitudes are reshaped into two groups of four, corresponding to the two classes. Squared amplitudes are summed within each group and normalised to obtain p_0 and p_1 .

Appendix B

MPS to VQC

B.1 Conversion of the Three-Site MPS into a Quantum Circuit

We now show how the three-site MPS contraction described in the previous section is embedded into the quantum circuit used in this work. The main idea is to rewrite each local MPS contraction as a matrix acting on the tensor product of the incoming bond vector and the local feature vector. The same parameterised orthogonal matrices are then used as quantum gates. The transfer of parameters is direct, although, as shown below, the complete VQC output is not in general identical to the classical MPS output because the VQC retains additional amplitudes that the middle MPS contraction does not keep.

Throughout this section, we consider the same three-site MPS with physical dimension $d = 2$, bond dimension $\chi = 4$, and output dimension 2. Since

$$\chi = 4 = 2^2, \tag{B.1}$$

the bond vector can be represented using two qubits, while each local feature vector can be represented using one qubit.

B.1.1 Isometric form of the MPS tensors

We assume that the MPS tensors are constructed from isometries. The orientation of the isometry depends on whether the tensor is taken from the columns or rows of the corresponding square matrix.

The first tensor has dimensions

$$A^{[1]} \in \mathbb{R}^{4 \times 2}. \tag{B.2}$$

We assume that its two columns are orthonormal, so that

$$(A^{[1]})^\top A^{[1]} = I_2. \quad (\text{B.3})$$

The first tensor is therefore a column isometry.

For the middle tensor, we combine the incoming bond index and the physical index into a single index. This produces a matrix

$$T^{[2]} \in \mathbb{R}^{4 \times 8}. \quad (\text{B.4})$$

We assume that its four rows are orthonormal:

$$T^{[2]} (T^{[2]})^\top = I_4. \quad (\text{B.5})$$

The final tensor used in the numerical implementation is different. The complete final matrix is retained,

$$U^{[3]} \in SO(8), \quad (\text{B.6})$$

and its row and column indices are factorised as

$$(o, e) \leftrightarrow \text{row}, \quad (\alpha, s) \leftrightarrow \text{column}. \quad (\text{B.7})$$

It is therefore reshaped into

$$A^{[3]} \in \mathbb{R}^{2 \times 4 \times 4 \times 2}. \quad (\text{B.8})$$

Thus, only the middle tensor is obtained by selecting rows of an 8×8 matrix. At the final site no row completion is required: the complete $SO(8)$ matrix is already used by both the MPS parameterisation and the corresponding quantum gate.

B.1.2 Conversion of the first tensor

Recall that the first classical contraction is

$$\mathbf{h}^{[1]} = A^{[1]} \boldsymbol{\phi}(x_1), \quad (\text{B.9})$$

where

$$\boldsymbol{\phi}(x_1) = \begin{pmatrix} \phi_0(x_1) \\ \phi_1(x_1) \end{pmatrix}. \quad (\text{B.10})$$

Since $A^{[1]}$ is a 4×2 column isometry, its two columns can be completed with two additional orthonormal columns to form an orthogonal matrix

$$U^{[1]} = \begin{pmatrix} A^{[1]} & C^{[1]} \end{pmatrix} \in SO(4), \quad (\text{B.11})$$

where $C^{[1]} \in \mathbb{R}^{4 \times 2}$ contains the additional columns.

The local feature vector is embedded into a four-dimensional vector by adding an auxiliary qubit prepared in the state $|0\rangle$:

$$|0\rangle \otimes |\phi(x_1)\rangle \longleftrightarrow \begin{pmatrix} \phi_0(x_1) \\ \phi_1(x_1) \\ 0 \\ 0 \end{pmatrix}. \quad (\text{B.12})$$

Applying $U^{[1]}$ gives

$$\begin{aligned} U^{[1]} \begin{pmatrix} \phi_0(x_1) \\ \phi_1(x_1) \\ 0 \\ 0 \end{pmatrix} &= \begin{pmatrix} A^{[1]} & C^{[1]} \end{pmatrix} \begin{pmatrix} \phi(x_1) \\ \mathbf{0} \end{pmatrix} \\ &= A^{[1]} \phi(x_1) \\ &= \mathbf{h}^{[1]}. \end{aligned} \quad (\text{B.13})$$

The additional columns $C^{[1]}$ do not contribute because the corresponding entries of the input vector are zero. Therefore, the first quantum gate produces exactly the same bond vector as the first classical MPS contraction.

In quantum-state notation,

$$U^{[1]} (|0\rangle \otimes |\phi(x_1)\rangle) = |h^{[1]}\rangle, \quad (\text{B.14})$$

where

$$|h^{[1]}\rangle = h_1^{[1]}|00\rangle + h_2^{[1]}|01\rangle + h_3^{[1]}|10\rangle + h_4^{[1]}|11\rangle. \quad (\text{B.15})$$

B.1.3 Rewriting the middle contraction

The middle MPS contraction was previously written component by component as

$$h_\beta^{[2]} = \sum_{\alpha=1}^4 \sum_{s=0}^1 A_{\beta\alpha s}^{[2]} h_\alpha^{[1]} \phi_s(x_2). \quad (\text{B.16})$$

We now rewrite this contraction as a single matrix-vector multiplication.

The incoming bond vector and the local feature vector are

$$\mathbf{h}^{[1]} = \begin{pmatrix} h_1^{[1]} \\ h_2^{[1]} \\ h_3^{[1]} \\ h_4^{[1]} \end{pmatrix}, \quad \phi(x_2) = \begin{pmatrix} \phi_0(x_2) \\ \phi_1(x_2) \end{pmatrix}. \quad (\text{B.17})$$

Their tensor product is

$$\mathbf{h}^{[1]} \otimes \boldsymbol{\phi}(x_2) = \begin{pmatrix} h_1^{[1]} \phi_0(x_2) \\ h_1^{[1]} \phi_1(x_2) \\ h_2^{[1]} \phi_0(x_2) \\ h_2^{[1]} \phi_1(x_2) \\ h_3^{[1]} \phi_0(x_2) \\ h_3^{[1]} \phi_1(x_2) \\ h_4^{[1]} \phi_0(x_2) \\ h_4^{[1]} \phi_1(x_2) \end{pmatrix} \in \mathbb{R}^8. \quad (\text{B.18})$$

This combines the incoming bond index α and the physical index s into the composite ordering

$$(\alpha, s) = (1, 0), (1, 1), (2, 0), (2, 1), (3, 0), (3, 1), (4, 0), (4, 1). \quad (\text{B.19})$$

Using the same ordering, the middle tensor is reshaped into the matrix

$$T^{[2]} = \begin{pmatrix} A_{110}^{[2]} & A_{111}^{[2]} & A_{120}^{[2]} & A_{121}^{[2]} & A_{130}^{[2]} & A_{131}^{[2]} & A_{140}^{[2]} & A_{141}^{[2]} \\ A_{210}^{[2]} & A_{211}^{[2]} & A_{220}^{[2]} & A_{221}^{[2]} & A_{230}^{[2]} & A_{231}^{[2]} & A_{240}^{[2]} & A_{241}^{[2]} \\ A_{310}^{[2]} & A_{311}^{[2]} & A_{320}^{[2]} & A_{321}^{[2]} & A_{330}^{[2]} & A_{331}^{[2]} & A_{340}^{[2]} & A_{341}^{[2]} \\ A_{410}^{[2]} & A_{411}^{[2]} & A_{420}^{[2]} & A_{421}^{[2]} & A_{430}^{[2]} & A_{431}^{[2]} & A_{440}^{[2]} & A_{441}^{[2]} \end{pmatrix}. \quad (\text{B.20})$$

The complete middle contraction can then be written as

$$\mathbf{h}^{[2]} = T^{[2]} (\mathbf{h}^{[1]} \otimes \boldsymbol{\phi}(x_2)). \quad (\text{B.21})$$

Writing the multiplication explicitly,

$$\begin{pmatrix} h_1^{[2]} \\ h_2^{[2]} \\ h_3^{[2]} \\ h_4^{[2]} \end{pmatrix} = T^{[2]} \begin{pmatrix} h_1^{[1]} \phi_0(x_2) \\ h_1^{[1]} \phi_1(x_2) \\ h_2^{[1]} \phi_0(x_2) \\ h_2^{[1]} \phi_1(x_2) \\ h_3^{[1]} \phi_0(x_2) \\ h_3^{[1]} \phi_1(x_2) \\ h_4^{[1]} \phi_0(x_2) \\ h_4^{[1]} \phi_1(x_2) \end{pmatrix}. \quad (\text{B.22})$$

For example, the first row produces

$$\begin{aligned}
h_1^{[2]} = & A_{110}^{[2]} h_1^{[1]} \phi_0(x_2) + A_{111}^{[2]} h_1^{[1]} \phi_1(x_2) \\
& + A_{120}^{[2]} h_2^{[1]} \phi_0(x_2) + A_{121}^{[2]} h_2^{[1]} \phi_1(x_2) \\
& + A_{130}^{[2]} h_3^{[1]} \phi_0(x_2) + A_{131}^{[2]} h_3^{[1]} \phi_1(x_2) \\
& + A_{140}^{[2]} h_4^{[1]} \phi_0(x_2) + A_{141}^{[2]} h_4^{[1]} \phi_1(x_2).
\end{aligned} \tag{B.23}$$

This is exactly the same scalar obtained from the row-vector-matrix-column- vector contraction in the previous section. The tensor product has only collected all products $h_\alpha^{[1]} \phi_s(x_2)$ into a single vector.

B.1.4 Orthogonal completion of the middle tensor

The matrix $T^{[2]}$ has dimensions 4×8 , so it cannot directly represent a quantum gate. A gate acting on three qubits must be an 8×8 unitary matrix.

Since the rows of $T^{[2]}$ are orthonormal, they can be completed with four additional orthonormal rows. We therefore define

$$U^{[2]} = \begin{pmatrix} T^{[2]} \\ C^{[2]} \end{pmatrix} \in SO(8), \tag{B.24}$$

where

$$C^{[2]} \in \mathbb{R}^{4 \times 8} \tag{B.25}$$

contains the additional rows required to complete the matrix.

The complete matrix satisfies

$$U^{[2]} (U^{[2]})^\top = (U^{[2]})^\top U^{[2]} = I_8. \tag{B.26}$$

Because $U^{[2]}$ is real, its transpose is equal to its conjugate transpose. It is therefore also unitary:

$$(U^{[2]})^\dagger U^{[2]} = I_8. \tag{B.27}$$

Hence, $U^{[2]}$ can be applied as a three-qubit quantum gate.

In our method, the unitary completion is not performed after training. Instead, the complete matrix $U^{[2]} \in SO(8)$ is parameterised directly, and the middle MPS tensor is formed from its first four rows:

$$T^{[2]} = U_{0:4,\cdot}^{[2]}. \tag{B.28}$$

The additional rows $C^{[2]}$ are already generated by the same parameterisation. This means that the parameters learned by the MPS can be transferred directly to the quantum gate without applying an intermediate decomposition or unitary-completion algorithm.

B.1.5 Action of the middle quantum gate

Applying the completed matrix to the tensor-product input gives

$$\begin{aligned} U^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)) &= \begin{pmatrix} T^{[2]} \\ C^{[2]} \end{pmatrix} (\mathbf{h}^{[1]} \otimes \phi(x_2)) \\ &= \begin{pmatrix} T^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)) \\ C^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)) \end{pmatrix}. \end{aligned} \quad (\text{B.29})$$

Using the definition of the classical contraction, this becomes

$$U^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)) = \begin{pmatrix} \mathbf{h}^{[2]} \\ \mathbf{g}^{[2]} \end{pmatrix}, \quad (\text{B.30})$$

where

$$\mathbf{g}^{[2]} = C^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)) \in \mathbb{R}^4. \quad (\text{B.31})$$

Therefore, the first four components produced by the quantum gate are exactly

$$\mathbf{h}^{[2]} = \begin{pmatrix} h_1^{[2]} \\ h_2^{[2]} \\ h_3^{[2]} \\ h_4^{[2]} \end{pmatrix}, \quad (\text{B.32})$$

which is the same bond vector produced by the classical MPS contraction.

The additional vector $\mathbf{g}^{[2]}$ is required because the quantum gate must act on the complete eight-dimensional space. It contains the amplitudes in the complementary subspace introduced by the orthogonal completion.

The selection of the MPS part of the output can be written using the projector

$$P_4 = \begin{pmatrix} I_4 & 0_{4 \times 4} \end{pmatrix}. \quad (\text{B.33})$$

We then have

$$\mathbf{h}^{[2]} = P_4 U^{[2]}(\mathbf{h}^{[1]} \otimes \phi(x_2)). \quad (\text{B.34})$$

B.1.6 Quantum-state interpretation of the middle contraction

The two-qubit bond state is

$$|h^{[1]}\rangle = h_1^{[1]}|00\rangle + h_2^{[1]}|01\rangle + h_3^{[1]}|10\rangle + h_4^{[1]}|11\rangle, \quad (\text{B.35})$$

and the local feature state is

$$|\phi(x_2)\rangle = \phi_0(x_2)|0\rangle + \phi_1(x_2)|1\rangle. \quad (\text{B.36})$$

The input to the middle gate is therefore

$$|h^{[1]}\rangle \otimes |\phi(x_2)\rangle. \quad (\text{B.37})$$

Applying $U^{[2]}$ gives

$$\begin{aligned} U^{[2]}(|h^{[1]}\rangle \otimes |\phi(x_2)\rangle) &= h_1^{[2]}|000\rangle + h_2^{[2]}|001\rangle + h_3^{[2]}|010\rangle + h_4^{[2]}|011\rangle \\ &\quad + g_1^{[2]}|100\rangle + g_2^{[2]}|101\rangle + g_3^{[2]}|110\rangle + g_4^{[2]}|111\rangle. \end{aligned} \quad (\text{B.38})$$

The amplitudes associated with the first qubit being in the state $|0\rangle$ are precisely the components of the next MPS bond vector:

$$|0\rangle \otimes |h^{[2]}\rangle. \quad (\text{B.39})$$

The remaining amplitudes lie in the complementary branch in which the first qubit is in the state $|1\rangle$. In the sequential circuit construction, the qubit containing this branch is no longer included in the next moving gate window. The relevant MPS contraction therefore remains stored in the computational-basis branch in which the discarded qubits are in the state $|0\rangle$.

B.1.7 Conversion of the final tensor

The last operation receives the four-dimensional MPS bond vector $\mathbf{h}^{[2]}$ and the two-dimensional feature vector $\phi(x_3)$. Their tensor product is

$$\mathbf{z}^{[3]} = \mathbf{h}^{[2]} \otimes \phi(x_3) \in \mathbb{R}^8. \quad (\text{B.40})$$

In the implementation, the complete final matrix

$$U^{[3]} \in SO(8) \quad (\text{B.41})$$

is used. Its column index is the combined input index (α, s) , while its row index is split into a class index $o \in \{0, 1\}$ and a four-dimensional auxiliary index $e \in \{1, 2, 3, 4\}$:

$$U_{(o,e),(\alpha,s)}^{[3]} \longleftrightarrow A_{oe\alpha s}^{[3]}. \quad (\text{B.42})$$

The final classical contraction is therefore

$$\mathbf{y} = U^{[3]} (\mathbf{h}^{[2]} \otimes \phi(x_3)) \in \mathbb{R}^8. \quad (\text{B.43})$$

The output is grouped according to the class index,

$$\mathbf{y} = \begin{pmatrix} y_{01} \\ y_{02} \\ y_{03} \\ y_{04} \\ y_{11} \\ y_{12} \\ y_{13} \\ y_{14} \end{pmatrix} = \begin{pmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \end{pmatrix}. \quad (\text{B.44})$$

The MPS class weights are

$$w_0 = \sum_{e=1}^4 |y_{0e}|^2, \quad w_1 = \sum_{e=1}^4 |y_{1e}|^2, \quad (\text{B.45})$$

and the final probabilities are

$$p_o = \frac{w_o}{w_0 + w_1}, \quad o \in \{0, 1\}. \quad (\text{B.46})$$

This is slightly different from the simpler two-logit construction in which only two rows of the final $SO(8)$ matrix are retained and the MPS directly produces two numbers y_0 and y_1 . The code used for the numerical results does not use those two logits. Instead, it keeps the full final matrix and produces eight amplitudes whose squared magnitudes are grouped into the two class probabilities.

The advantage of this form is that it matches the probability measurement of the final three-qubit gate naturally. If the first wire in the final gate window is identified as the class qubit, the remaining two wires label the four values of the auxiliary index e . Within the MPS branch, the corresponding class weights are

$$w_0 = \sum_{e=1}^4 |y_{0e}|^2, \quad w_1 = \sum_{e=1}^4 |y_{1e}|^2. \quad (\text{B.47})$$

These weights have the same Born-rule form as measuring the class qubit and marginalising over the other two qubits of the final $SO(8)$ block. Since the retained MPS branch is not necessarily normalised, the corresponding class probabilities are obtained by normalising these weights as in Eq. B.46.

B.1.8 The complete VQC contains additional amplitudes

The previous equation describes the final gate acting on the MPS bond branch. The complete VQC, however, contains more amplitudes because the middle quantum gate uses all rows of $U^{[2]}$, whereas the MPS keeps only the first four rows.

For the three-site example, $\chi = 4$ requires two bond qubits. Together with three data sites, the circuit contains

$$n_q = 3 + \log_2(4) - 1 = 4 \quad (\text{B.47})$$

qubits. We label them q_0, q_1, q_2, q_3 . The first gate acts on (q_0, q_1) , the middle gate acts on (q_0, q_1, q_2) , and the final gate acts on (q_1, q_2, q_3) . The class wire is q_1 , the first wire of the final gate window.

After the middle gate, the state on (q_0, q_1, q_2) can be written as

$$|\Psi^{[2]}\rangle = |0\rangle_{q_0}|h^{[2]}\rangle + |1\rangle_{q_0}|g^{[2]}\rangle. \quad (\text{B.48})$$

The first branch contains the four amplitudes retained by the MPS. The second branch contains the four complementary amplitudes generated by the remaining rows of the full orthogonal matrix. The classical MPS continues only with $\mathbf{h}^{[2]}$, while the quantum circuit must retain both branches.

The final gate does not act on q_0 . Therefore, before any additional extension gates are applied, the four-qubit output can be written schematically as

$$|\Psi_{\text{VQC}}\rangle = |0\rangle_{q_0} \sum_{o,e} y_{oe} |o, e\rangle + |1\rangle_{q_0} \sum_{o,e} \tilde{y}_{oe} |o, e\rangle, \quad (\text{B.49})$$

where y_{oe} are the eight amplitudes obtained from the MPS branch and $\tilde{y}_{oe}$ are the eight amplitudes obtained from the complementary branch. The complete VQC therefore contains 16 amplitudes, while the final MPS contraction contains 8.

The VQC classification does not discard the complementary branch. Only the class wire q_1 is measured, so all other wires are marginalised over. For class 0,

$$P_{\text{VQC}}(0) = \sum_{q_0, q_2, q_3} |c_{q_0 0 q_2 q_3}|^2. \quad (\text{B.50})$$

Equivalently,

$$P_{\text{VQC}}(0) = \sum_{e=1}^4 |y_{0e}|^2 + \sum_{e=1}^4 |\tilde{y}_{0e}|^2. \quad (\text{B.51})$$

Similarly,

$$P_{\text{VQC}}(1) = \sum_{e=1}^4 |y_{1e}|^2 + \sum_{e=1}^4 |\tilde{y}_{1e}|^2. \quad (\text{B.52})$$

The first term in each expression is the contribution associated with the MPS branch. The second term comes from amplitudes which the classical MPS did not retain but which are necessarily present in the complete unitary evolution.

If we condition the circuit on the retired wire being in the MPS branch, $q_0 = 0$, then

$$P_{\text{VQC}}(q_1 = o \mid q_0 = 0) = \frac{\sum_{e=1}^4 |y_{oe}|^2}{\sum_{o'=0}^1 \sum_{e=1}^4 |y_{o'e}|^2} = p_{\text{MPS}}(o). \quad (\text{B.53})$$

Hence, before the optional extension gates, the MPS probability is recovered as a conditional probability of the quantum circuit. The actual VQC used for classification does not perform this postselection. It measures only the class wire, and therefore the complementary branch also contributes to the measured probability.

For more sites, the same structure repeats. Each middle MPS contraction keeps only the selected χ -dimensional output of its 2χ -dimensional parent matrix, whereas the VQC retains the complementary amplitudes. Each wire which leaves the moving gate window labels another branch of the full quantum state.

For example, with four sites and $\chi = 4$, the circuit contains

$$n_q = 4 + 2 - 1 = 5 \quad (\text{B.54})$$

qubits and therefore $2^5 = 32$ amplitudes. The two retired wires label four eight-dimensional sectors. The MPS contraction corresponds to the sector in which both retired wires are 0, while the VQC class measurement also includes the other three sectors.

B.1.9 Relation between the MPS and VQC outputs

The trained MPS parameters directly determine the matrices $U^{[1]}, U^{[2]}, U^{[3]}$, and the same angle tensors are copied to the corresponding VQC gates. In this sense, the parameter transfer is exact and does not require an SVD, an approximate matrix decomposition, or a separate unitary completion step.

The equality is not, however, an equality between the complete MPS and VQC classification outputs. The MPS uses selected rows at the middle sites and then normalises the class weights obtained from its retained branch. The quantum circuit evolves the complete orthogonal matrices and measures only the class wire, thereby including all unmeasured branches in the marginal probability.

This distinction is also why the present implementation should not be described as producing the same two logits as the simpler two-logit derivation. The final MPS used here produces eight amplitudes, grouped into two probabilities, while the VQC produces a larger global state and obtains its two class probabilities by measuring one qubit.

The purpose of the MPS-to-VQC construction is therefore not to make every output amplitude identical. Its purpose is to learn a structured set of local orthogonal transformations classically and transfer the corresponding parameters directly to the quantum circuit, giving the VQC a pretrained initialisation.

When the optional near-identity extension is enabled, additional trainable two-qubit $SO(4)$ gates are applied after the MPS-derived gates. These gates are initialised close to the identity but can mix the different sectors of the quantum state. The branch-wise relations above should therefore be understood for the MPS-derived circuit before these extension gates, or in the limit in which the extension gates are exactly the identity.

Bibliography

- [1] Y Du, X Wang, N Guo, Z Yu, Y Qian, K Zhang, M Hsieh, P Rebentrost and D Tao. *2025 Quantum Machine Learning: A Hands-on Tutorial for Machine Learning Practitioners and Researchers*. arXiv preprint, arXiv:2502.01146.
- [2] M A Nielsen and I L Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011..
- [3] D M Greenberger, M A Horne and A Zeilinger. *Going Beyond Bell's Theorem*. In M Kafatos (ed.), *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, Kluwer Academic, pp 69–72, 1989.
- [4] R Jozsa and N Linden. *2003 On the role of entanglement in quantum-computational speed-up*. Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences, Volume 459, Issue 2036, pp 2011–2032.
- [5] T M Mitchell. *1997 Machine Learning*. McGraw-Hill, New York.
- [6] I Goodfellow, Y Bengio and A Courville. *2016 Deep Learning*. MIT Press.
- [7] Y LeCun, L Bottou, Y Bengio and P Haffner. *1998 Gradient-based learning applied to document recognition*. Proceedings of the IEEE, Volume 86, Number 11, pp 2278–2324.
- [8] C M Bishop. *2006 Pattern Recognition and Machine Learning*. Springer, New York.
- [9] E M Stoudenmire and S R White. *2016 Supervised learning with quantum-inspired tensor networks*. Advances in Neural Information Processing Systems, Volume 29, pp 4799–4807.
- [10] D P Kingma and J Ba. *2015 Adam: A method for stochastic optimization*. 3rd International Conference on Learning Representations (ICLR). arXiv preprint, arXiv:1412.6980.
- [11] L Prechelt. *1998 Automatic early stopping using cross validation: quantifying the criteria*. Neural Networks, Volume 11, Issue 4, pp 761–767.
- [12] M Raghu, B Poole, J Kleinberg, S Ganguli and J Sohl-Dickstein. *2017 On the expressive power of deep neural networks*. International Conference on Machine Learning (ICML), PMLR, Volume 70, pp 2847–2854.

- [13] R Penrose. *1971 Applications of negative dimensional tensors*. Combinatorial Mathematics and Its Applications (ed. D Welsh), Academic Press, New York, pp 221–244.
- [14] S R White. *1992 Density matrix formulation for quantum renormalization groups*. Physical Review Letters, Volume 69, Issue 19, pp 2863–2866.
- [15] S Östlund and S Rommer. *1995 Thermodynamic limit of density matrix renormalization*. Physical Review Letters, Volume 75, Issue 19, pp 3537–3540.
- [16] F Verstraete, V Murg and J I Cirac. *2008 Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems*. Advances in Physics, Volume 57, Issue 2, pp 143–224.
- [17] R Orús. *2014 A practical introduction to tensor networks: Matrix product states and projected entangled pair states*. Annals of Physics, Volume 349, pp 117–158.
- [18] U Schollwöck. *2011 The density-matrix renormalization group in the age of matrix product states*. Annals of Physics, Volume 326, Issue 1, pp 96–192.
- [19] G Vidal. *2003 Efficient classical simulation of slightly entangled quantum computations*. Physical Review Letters, Volume 91, Issue 14, p 147902.
- [20] I V Oseledets. *2011 Tensor-train decomposition*. SIAM Journal on Scientific Computing, Volume 33, Issue 5, pp 2295–2317.
- [21] S Efthymiou, J Hidary and S Leichenauer. *2019 TensorNetwork for Machine Learning*. arXiv preprint, arXiv:1906.06329.
- [22] E. Guo and D. Draper, *Representation Theorem for Matrix Product States*, arXiv:2103.08277, 2021.
- [23] A Mossi, B Žunkovič and K Flouris. *2024 A matrix product state model for simultaneous classification and generation*. arXiv preprint, arXiv:2406.17441.
- [24] J Biamonte, P Wittek, N Pancotti, P Rebentrost, N Wiebe and S Lloyd. *2017 Quantum machine learning*. Nature, Volume 549, Issue 7671, pp 195–202.
- [25] M Schuld, I Sinayskiy and F Petruccione. *2015 An introduction to quantum machine learning*. Contemporary Physics, Volume 56, Issue 2, pp 172–185.
- [26] V Havlíček, A D Córcoles, K Temme, A W Harrow, A Kandala, J M Chow and J M Gambetta. *2019 Supervised learning with quantum-enhanced feature spaces*. Nature, Volume 567, Issue 7747, pp 209–212.
- [27] K Beer, D Bondarenko, T Farrelly, T J Osborne, R Salzmann, D Scheiermann and R Wolf. *2020 Training deep quantum neural networks*. Nature Communications, Volume 11, p 808.
- [28] I Cong, S Choi and M D Lukin. *2019 Quantum convolutional neural networks*. Nature Physics, Volume 15, Issue 12, pp 1273–1278.

- [29] M Cerezo, A Sone, T Volkoff, L Cincio and P J Coles. *2021 Cost landscape of variational quantum algorithms brings barren plateaus for random parameterized quantum circuits*. Nature Communications, Volume 12, p 1791.
- [30] M Benedetti, E Lloyd, S Sack and M Fiorentini. *2019 Parameterized quantum circuits as machine learning models*. Quantum Science and Technology, Volume 4, Issue 4, p 043001.
- [31] M Cerezo, G Verdon, H-Y Huang, L Cincio and P J Coles. *2022 Challenges and opportunities in quantum machine learning*. Nature Computational Science, Volume 2, Issue 9, pp 567–576.
- [32] M Schuld. *2021 Supervised quantum machine learning models are kernel methods*. Physical Review A, Volume 103, Issue 3, p 032408.
- [33] J R McClean, S Boixo, V N Smelyanskiy, R Babbush and H Neven. *2018 Barren plateaus in quantum neural network training landscapes*. Nature Communications, Volume 9, p 4812.
- [34] J Kübler, A Arrasmith, L Cincio and P J Coles. *2020 An adaptive optimizer for measurement-frugal variational algorithms*. Quantum, Volume 4, p 263.
- [35] M Schuld, V Bergholm, C Gogolin, J Izaac and N Killoran. *2019 Evaluating analytic gradients on quantum hardware*. Physical Review A, Volume 99, Issue 3, p 032331.
- [36] M S Rudolph, J Miller, D Motlagh, J Chen, A Acharya and A Perdomo-Ortiz. *2023 Synergistic pretraining of parametrized quantum circuits via tensor networks*. Nature Communications, Volume 14, p 8367.
- [37] S-J Ran. *2020 Encoding of matrix product states into quantum circuits of one- and two-qubit gates*. Physical Review A, Volume 101, Issue 3, p 032310.
- [38] M Schuld and N Killoran. *2019 Quantum Machine Learning in Feature Hilbert Spaces*. Physical Review Letters, Volume 122, Issue 4, p 040504.
- [39] B Huggins, P Patil, B Mitchell, K B Whaley and E M Stoudenmire. *2019 Towards quantum machine learning with tensor networks*. Quantum Science and Technology, Volume 4, Issue 2, p 024001.
- [40] E Knill, G Ortiz and R D Somma. *2007 Optimal quantum measurements of expectation values of observables*. Physical Review A, Volume 75, Issue 1, p 012328.
- [41] V V Shende, S S Bullock and I L Markov. *2006 Synthesis of quantum-logic circuits*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, Volume 25, Issue 6, pp 1000–1010.